JSPM UNIVERSITY, PUNE
FACULTY OF SCIENCE AND TECHNOLOGY

SCHOOL OF COMPUTATIONAL SCIENCES

Introduction to Embedded Systems Lab Manual (230GETB41)

B.Tech in _____

Name: _____

PRN: _____Roll No: _____

Branch: _____Division: _____

2025 - 2026

JSPM University Pune B.Tech. "CSE" V/VII Sem		
Course Type: MMC	**Lab Course Title: Introduction to Embedded System Lab**	
Course Code: **230GETB41**	**Teaching Scheme:** **(Hrs./Week)**	**Examination Scheme:**
Credits: 1	**Lecture (L):** **Tutorial (T):** **Practical (P): 2** **Experiential Learning (EL):**	**Theory (TH): --** **Practical (PR): 50 Marks** **Oral (OR): 50 Marks**
Prerequisite Courses, if any: -		

List of Laboratory Experiments

[Conduct the following experiments by writing program using 8051/AT89C51 using an evaluation board/simulator and the required software tool]

1	Write a program to find the GCD and LCM of a given two bytes in a memory locations
2	Write a program to convert the given BCD number into its equivalent seven segment value.
3	Toggling of p1.5 with the delay of 5ms without interrupts using timer 1
4	Toggling of p1.2 with the delay of 5ms with interrupts using timer 0
5	Count no. 0f pulses without interrupts using counter 0
6	Count no. 0f pulses with interrupts using counter 1
7	Program to transfer data serially with baud rate of 9600
8	Program to receive data serially with baud rate of 9600
9	Sequence Generator Using Serial Interface in 8051
10	Programming for Delay Generation and Effect of CPU Clock
11	Programming with On-Chip Timers/Counters
12	Programming with On-Chip ADC
13	Programming for PWM Generation
14	Programming for I/O Interfacing (LED and Switch Interfacing)

Virtual LAB Links:

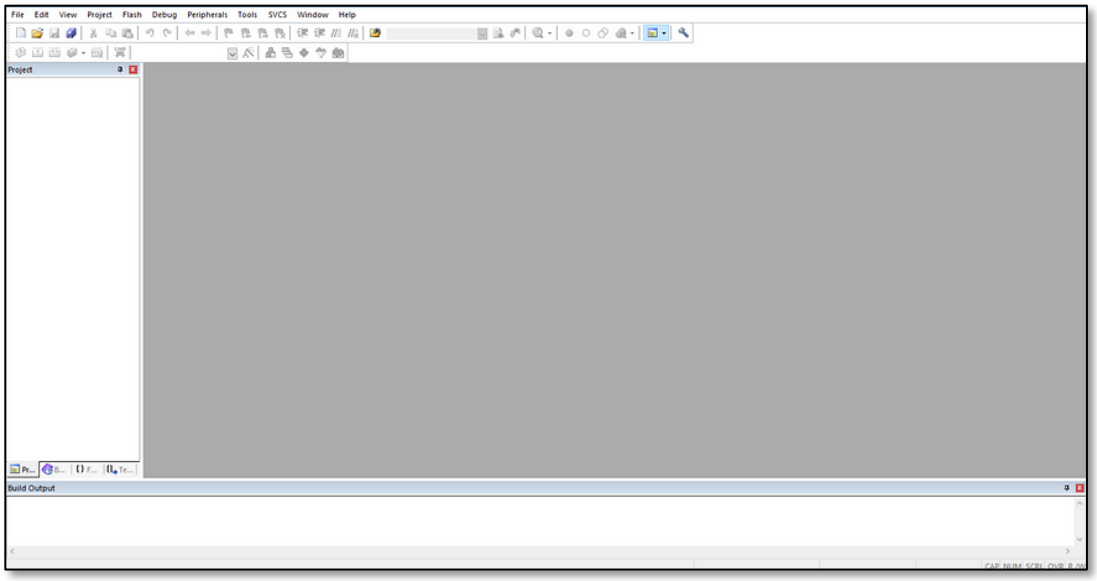
1. **Lab Name: Embedded Systems Link of the Virtual Lab:** https://esd-coep.vlabs.ac.in/Introduction.html
2. **Lab Name: Real Time Embedded Systems Laboratory**
   Link of the Virtual Lab: http://vlabs.iitkgp.ac.in/rtes/#

INTRODUCTION TO KEIL µvision

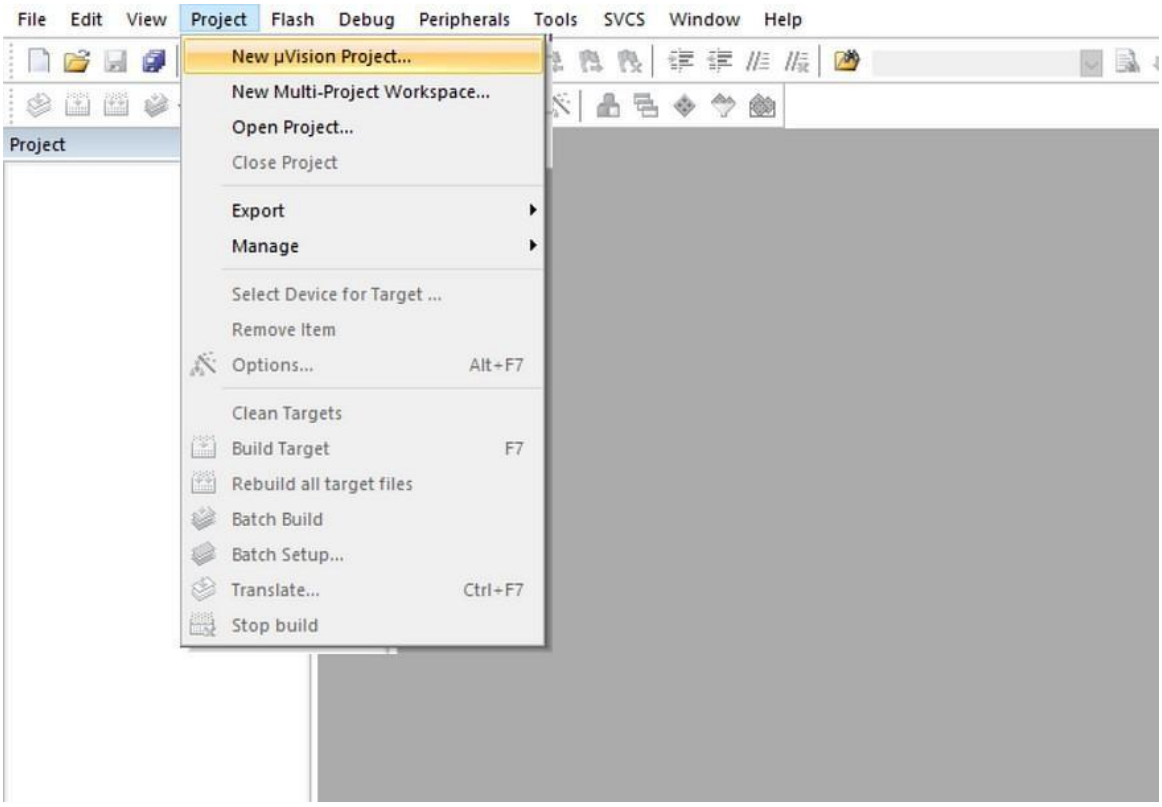
We will explore how to program the 8051 microcontroller using Keil uVision5.

Creating a Project in Keil µvision

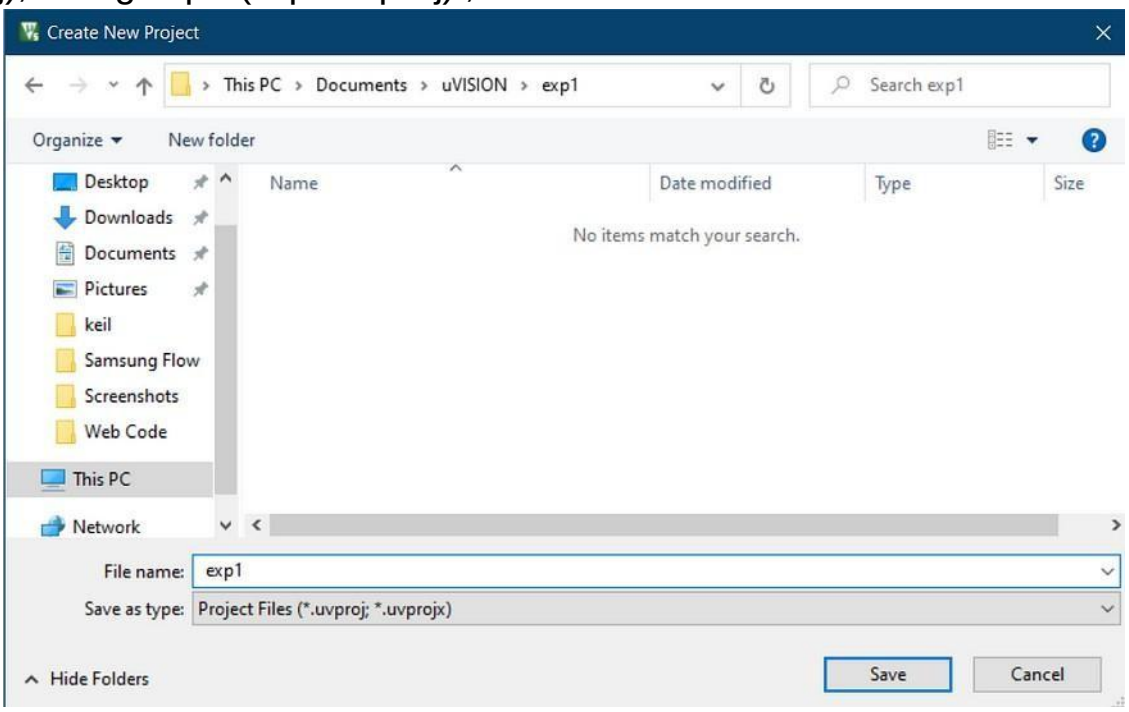
When we open Keil µvision for first time we will be able to see ide as shown in figure below.



To create a new 8051 project using Keil IDE, click on the 'Project 'item on the IDE Menu bar and select 'New uVision Project… 'as shown in the above image.

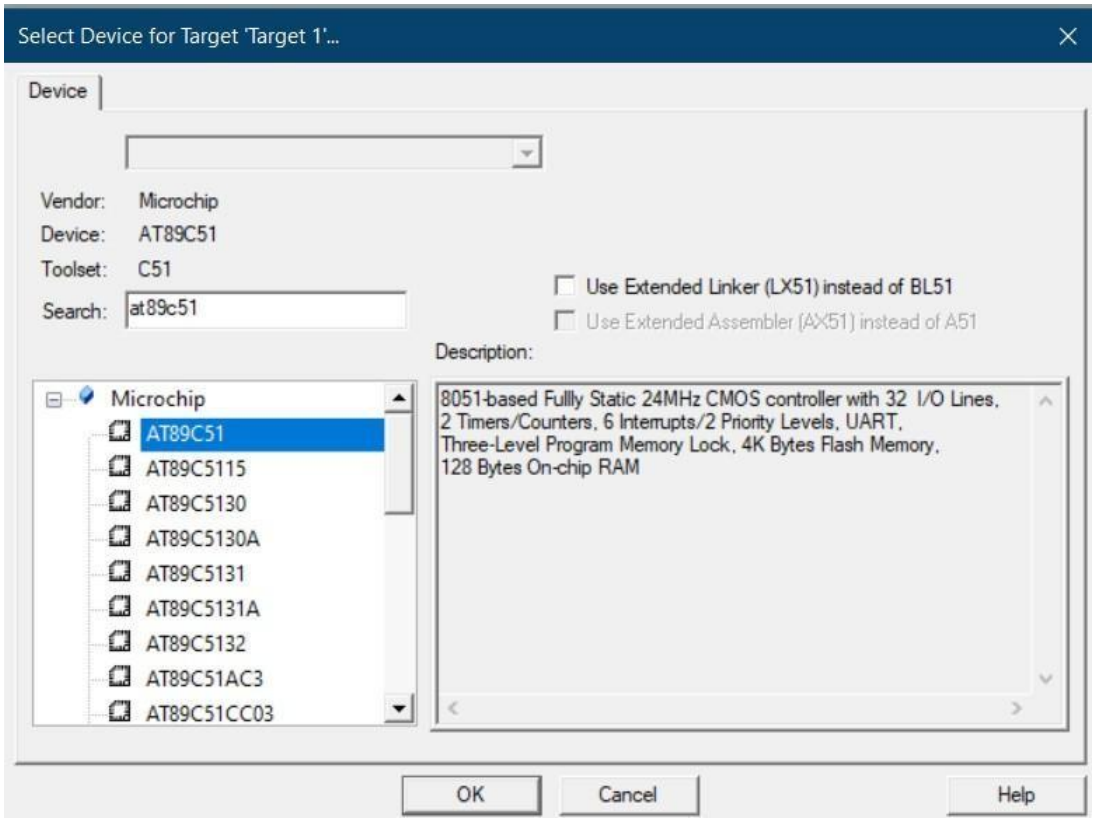


Now create a Folder to store your project and give a name to your Project files (*.uvproj), for eg exp1 (exp1.uvproj) , then save it.

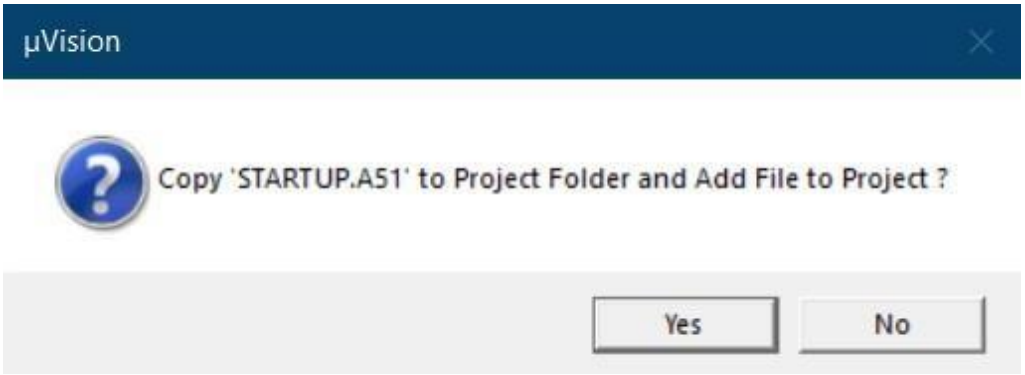


Selecting 8051 Microcontroller

- After saving the project , a dialog box will appear to select a Device
- Then type “at89c51” in search box
- Click on AT89C51 and click on OK

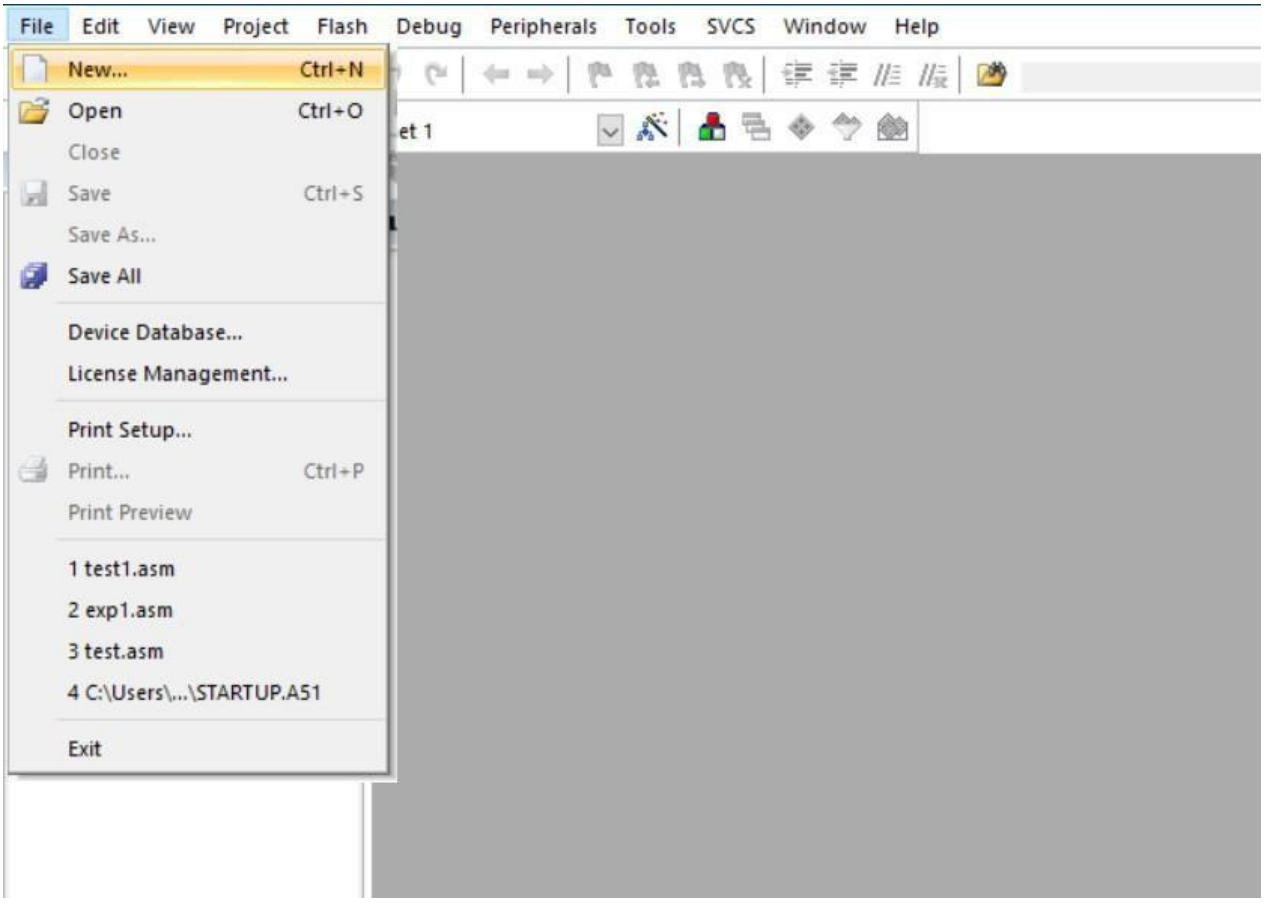


You will get another dialog as shown below. Asking to copy STARTUP.A51, Click on No

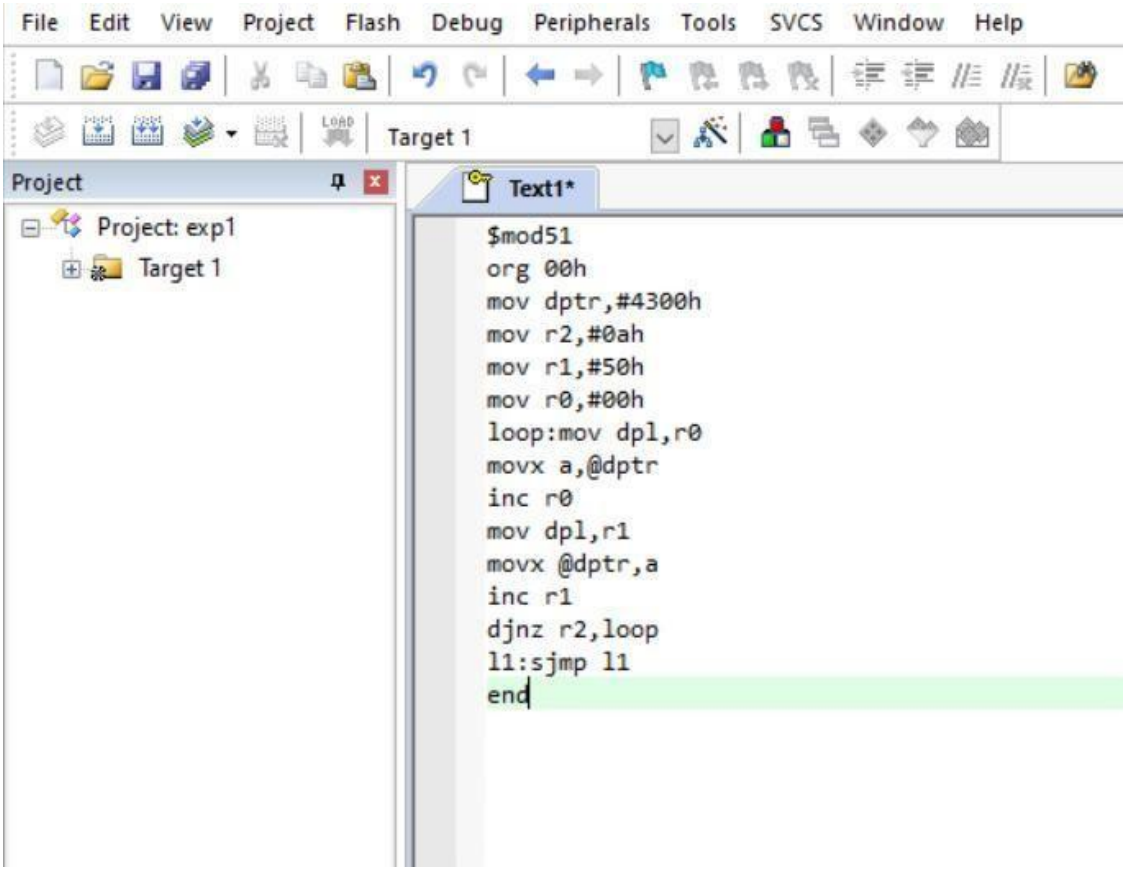


Writing Code

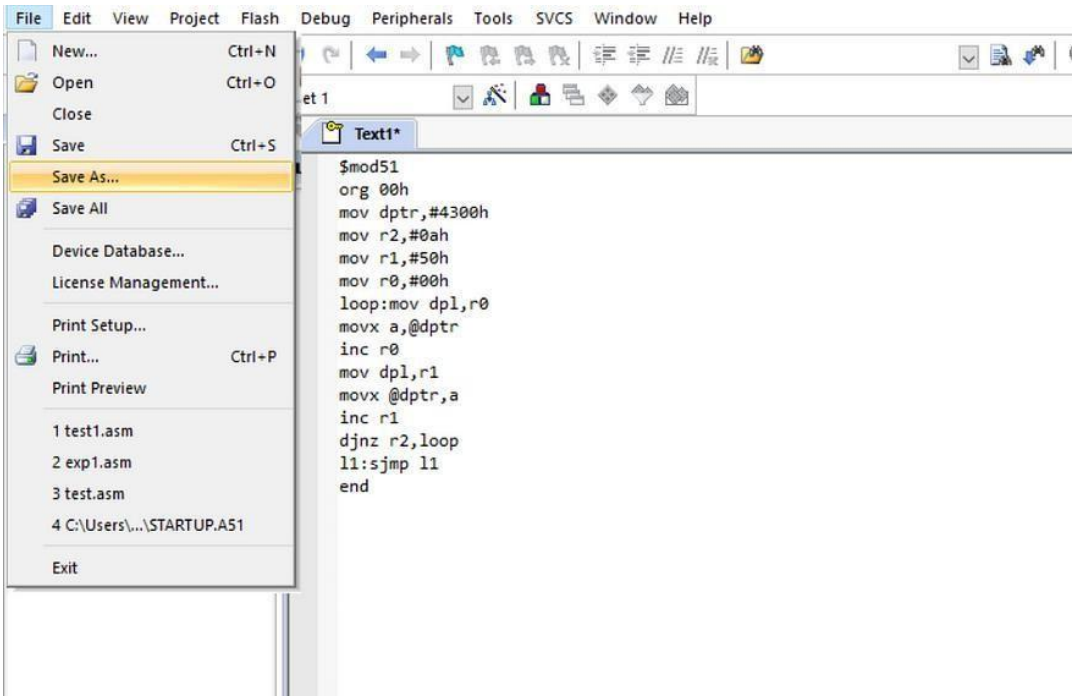
Now in order to write code we need a blank file, Click on File >> New... or can use keyboard shortcut Ctrl+ N



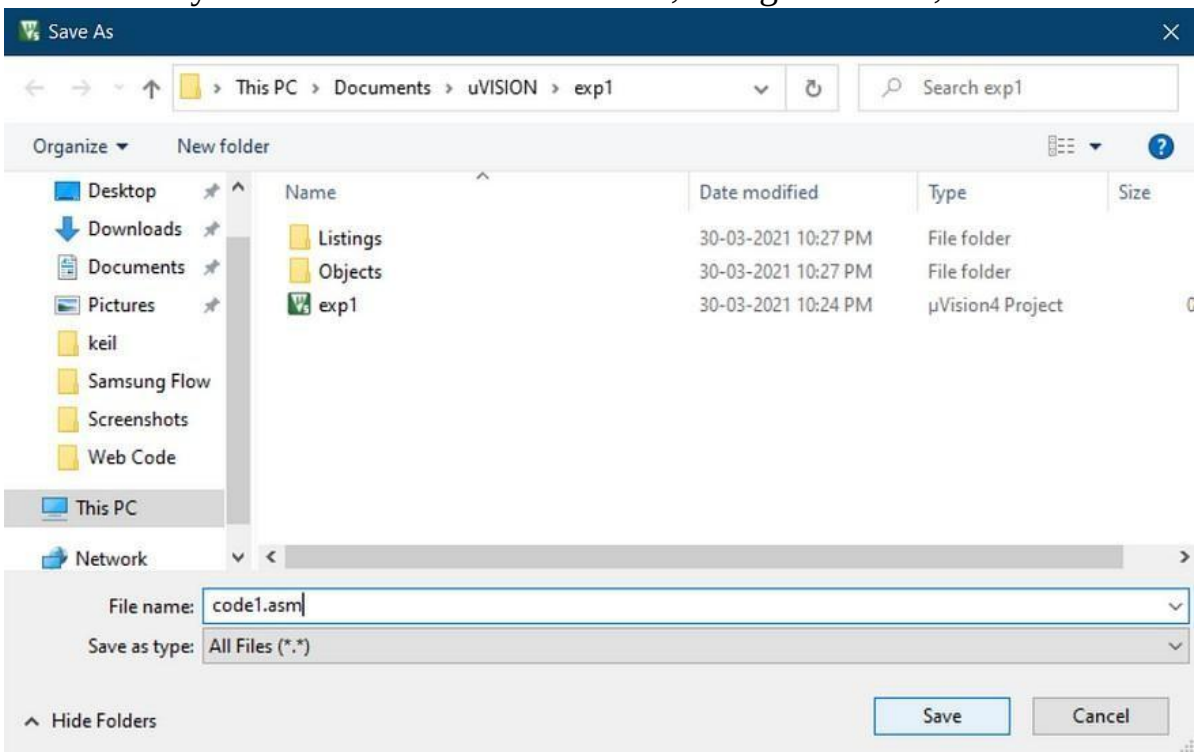
Write a code as shown in the figure below



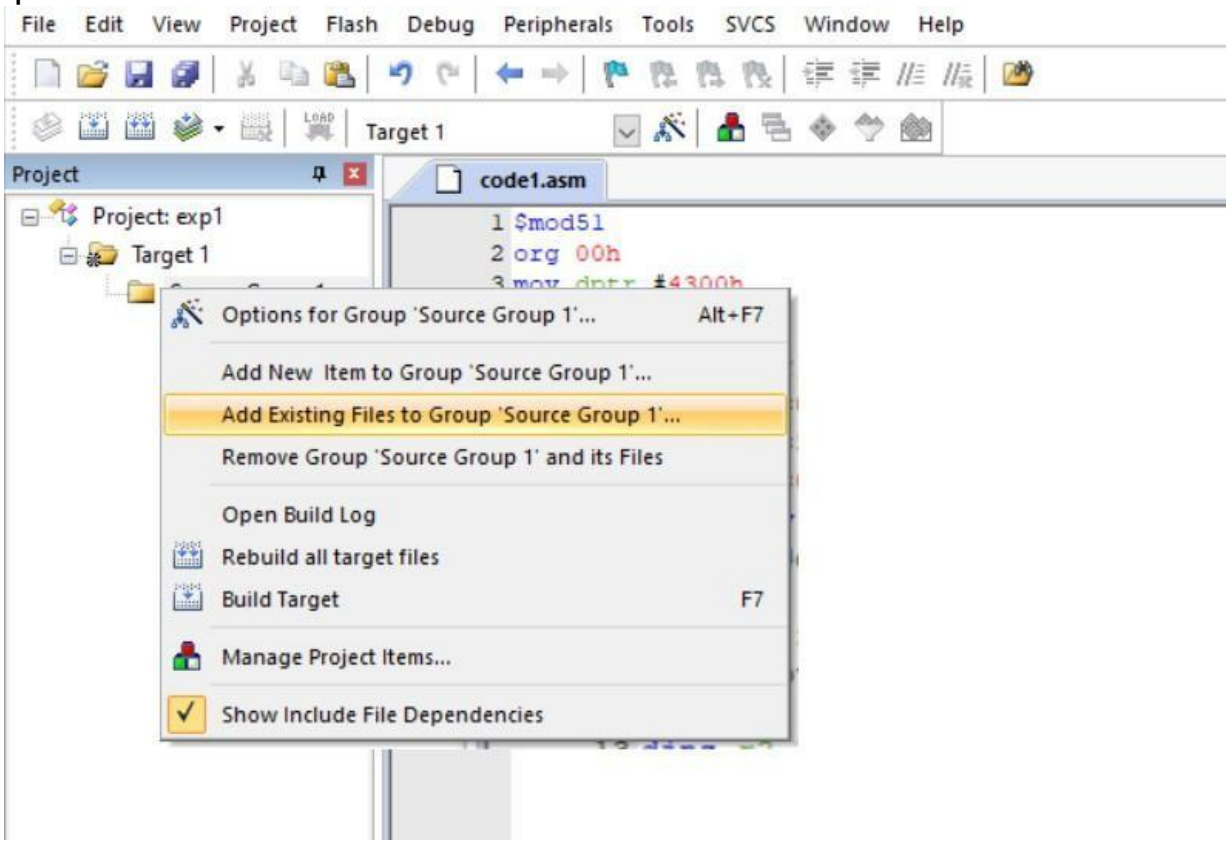
Now “Save As” the file , Click on File >> Save As...



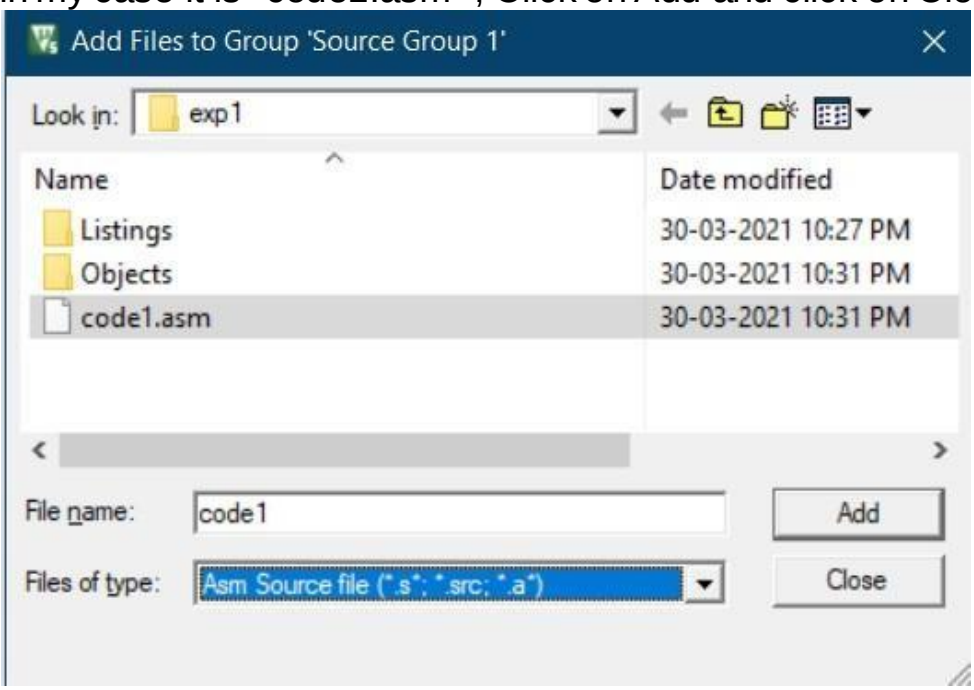
Give a file name to your code with extenstion *.asm, for eg code.asm , then save it.



In left corner you can see a Project Box , in that Project Box expand Target1 , then Right click on Source Group 1, Then click on Add Existing Files to Group 'Source Group 1'...

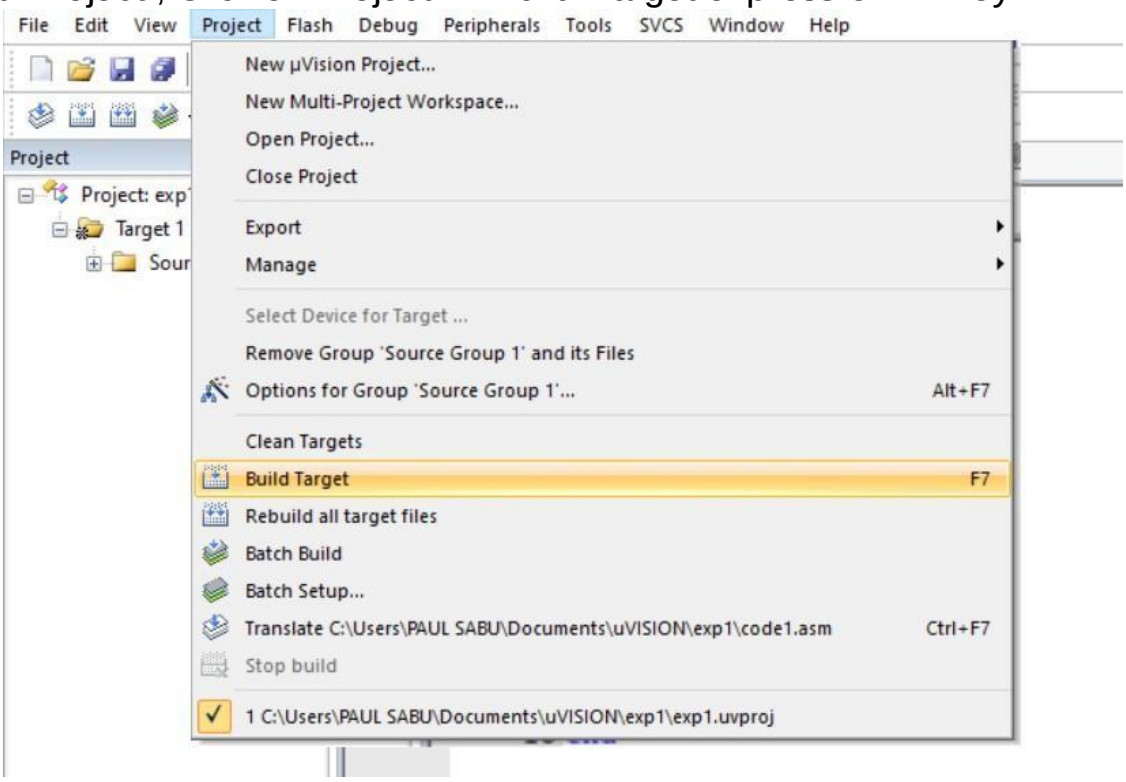


Select the Files of type : Asm Source file (*.s*;*.src*;*.a*) , Select the file with extenstion *.asm , here in my case it is "code1.asm" , Click on Add and click on Close

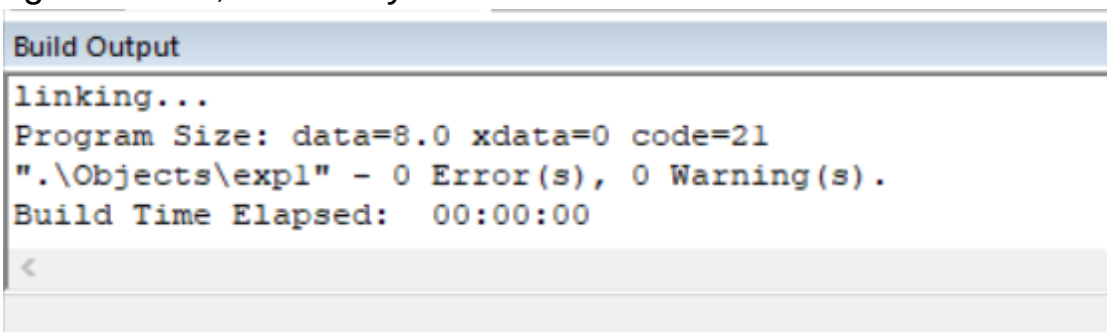


Project Build

To Bulid Project , Click on Project >> Build Target or press on F7 key

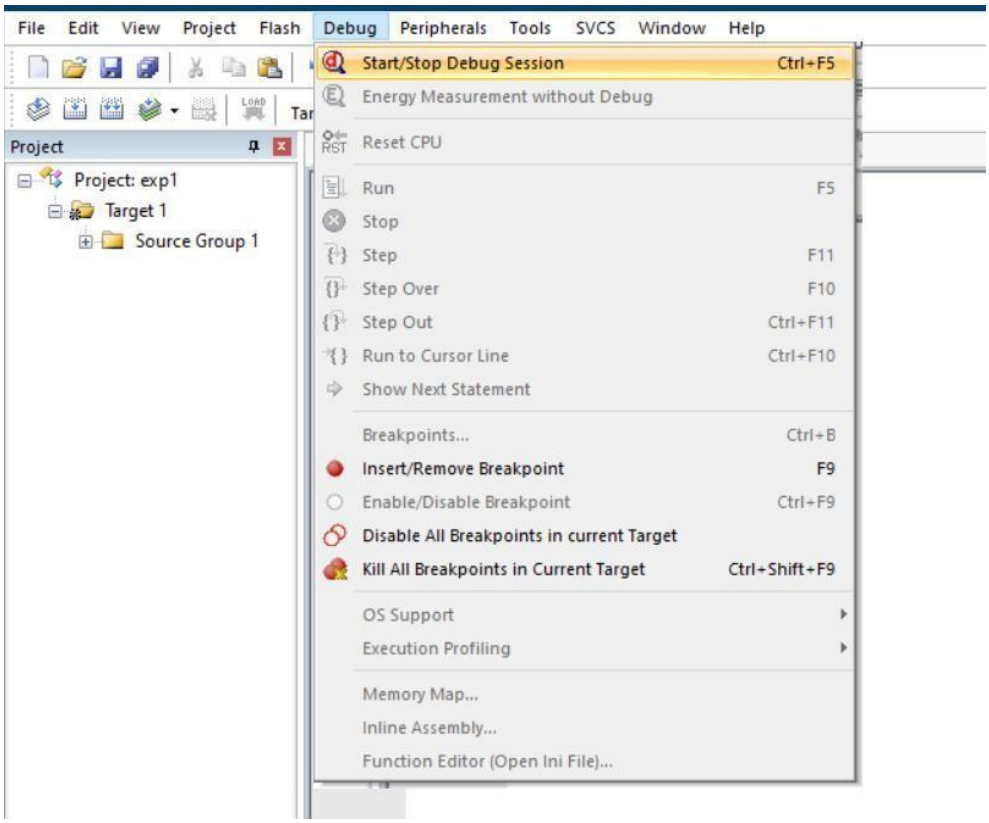


If the Build is successful there will not be any error or any warning as shown in the figure below , else verify the code and rebuild it.

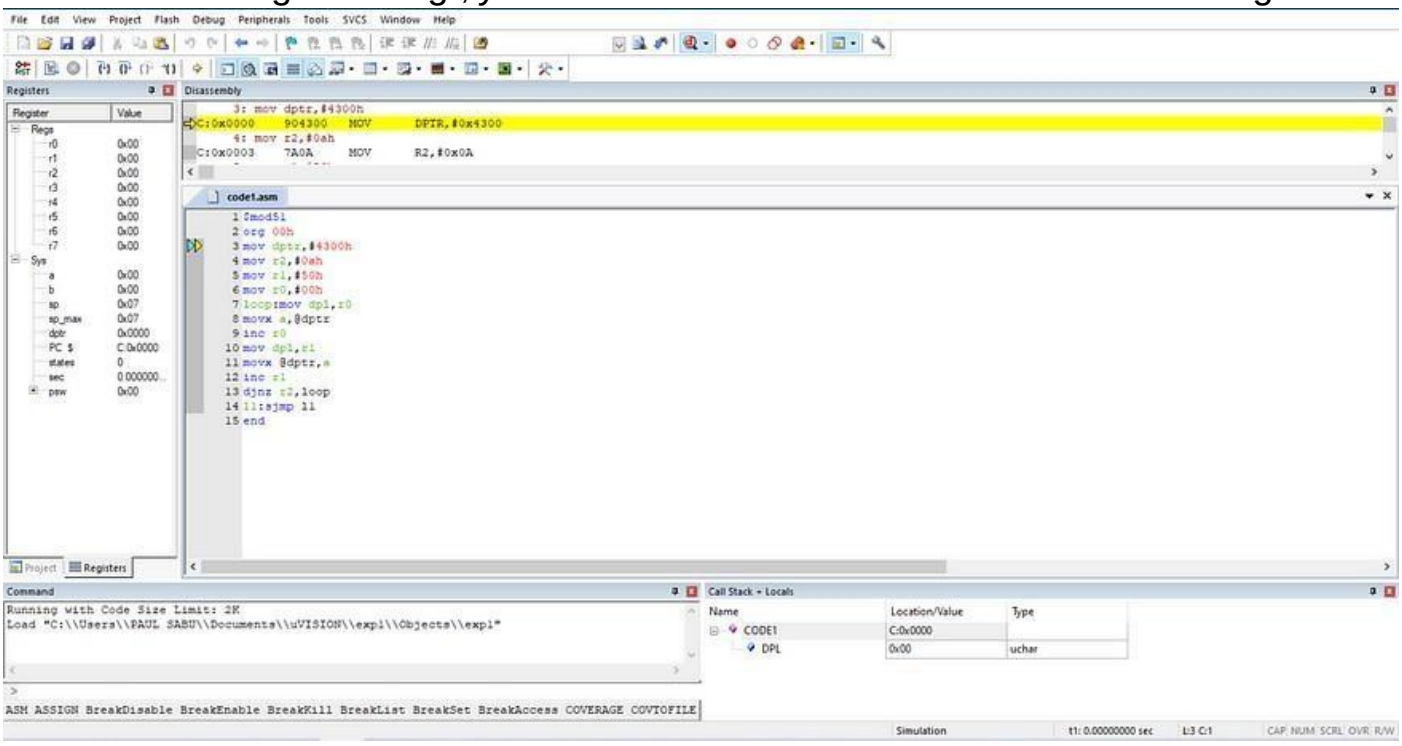


Debug and Run Code

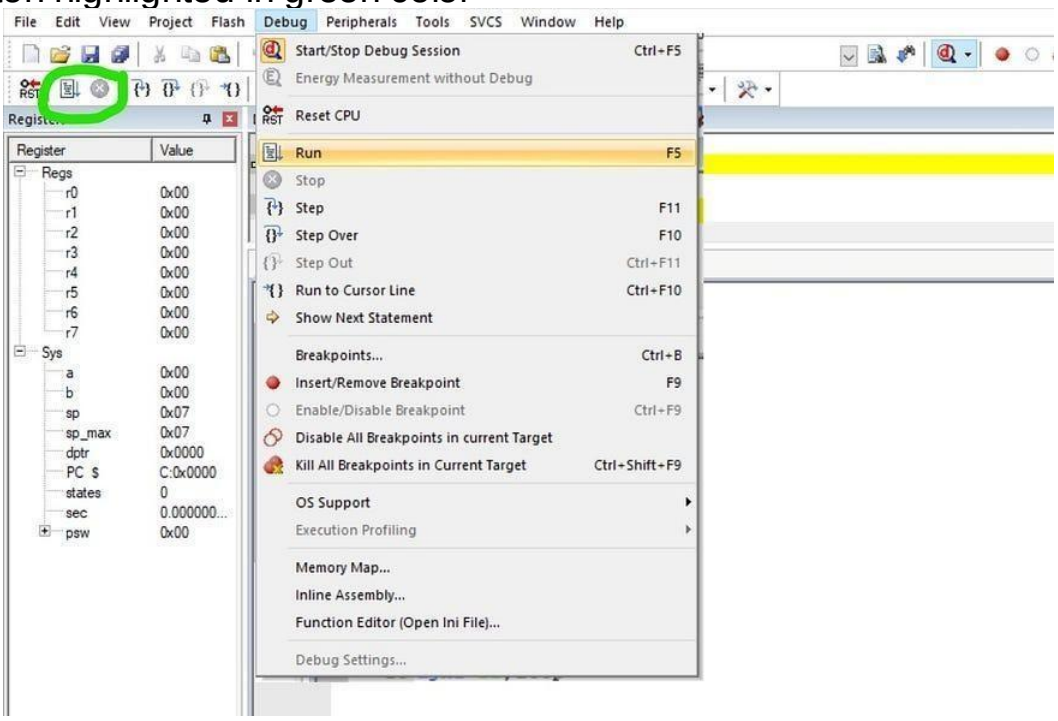
Next step is to debug the code, For that click on Debug >> Start/StopDebugSession or press on Ctrl+ F5



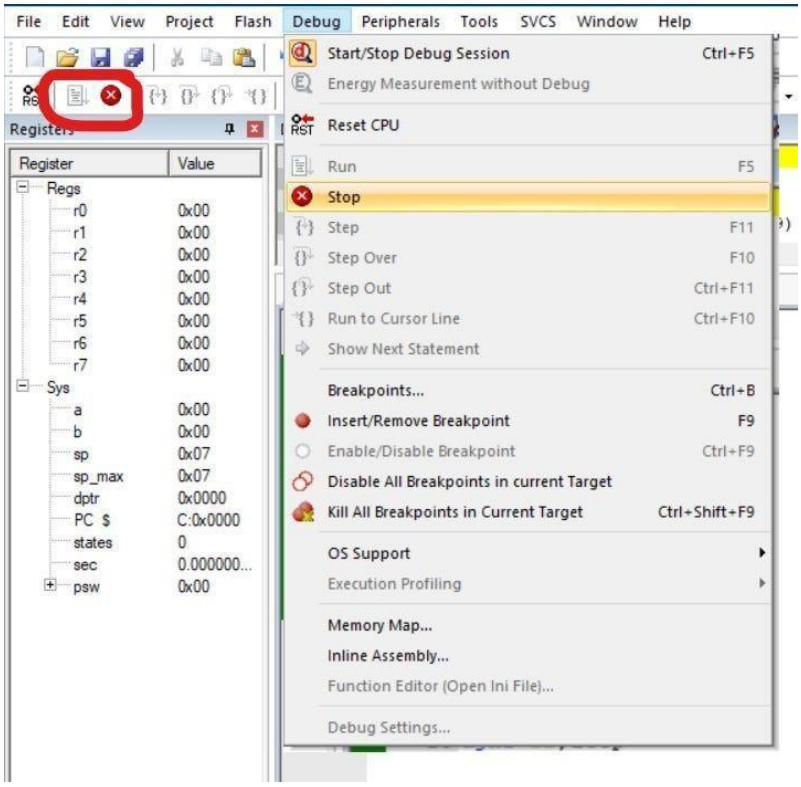
After Entering to debug , you will have a similar screen as shown in the figure



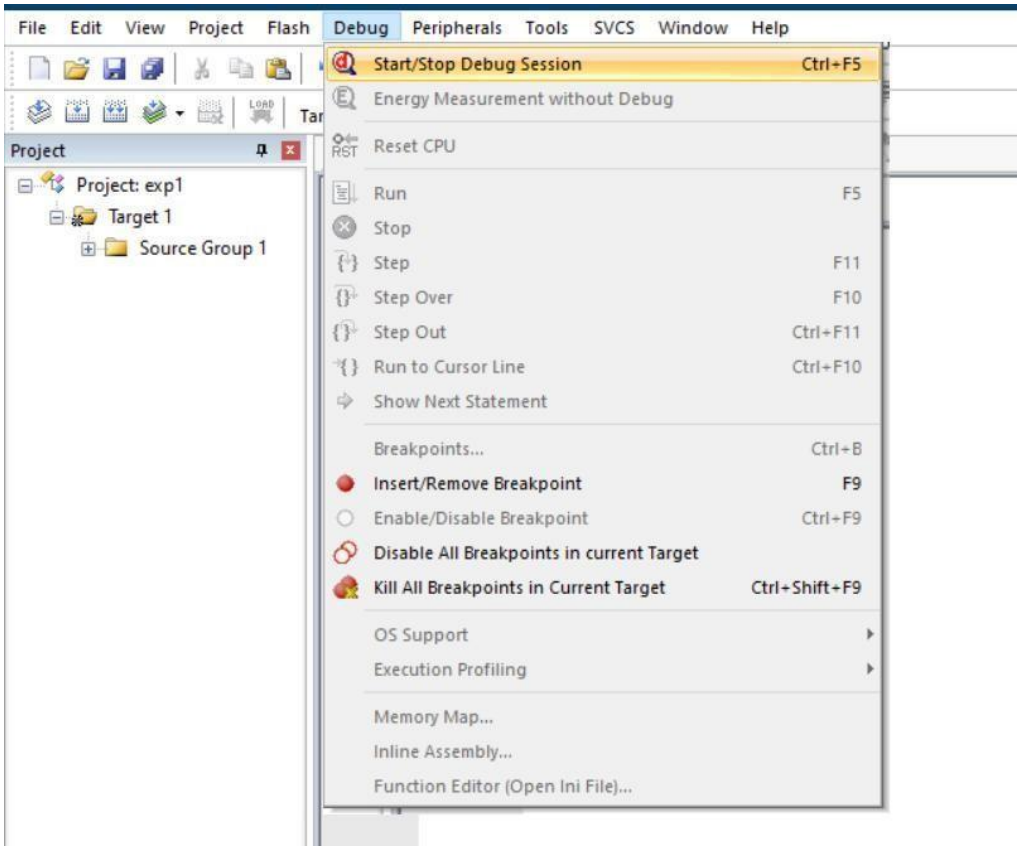
Next step is to Run the code , For that click on Debug >> Run or press on F5 or click "Run" button highlighted in green color



After a few seconds we need to Stop running the code , For that click on Debug >> Stop or click "Stop" button highlighted in red color



Before Closing the Keil uVision We need to stop Debugging, for that click on Debug >> Start/Stop Debug Session or press on Ctrl+ F5, Then close it



Experiment - 1

AIM : ***Write an assembly language program to find the GCD and LCM of a given two bytes in a memory locations.***

SOFTWARE REQUIRED: Keil µvision

ALGORITHM:

Algorithm for GCD (Greatest Common Divisor)
Let the two input numbers be A and B.
Step 1: Start
Step 2: Read two positive integers A and B
Step 3: Repeat the following until B becomes 0:

- a. Find the remainder R = A mod B
- b. Assign A = B
- c. Assign B = R

Step 4: When B = 0, the value of A is the GCD
Step 5: Print GCD
Step 6: Stop

Example (GCD of 48 and 18)

A	B	A mod B (R)
48	18	12
18	12	6
12	6	0

GCD = 6

Algorithm for LCM (Least Common Multiple)
Step 1: Start
Step 2: Read two positive integers A and B
Step 3: Find GCD of A and B using the GCD algorithm above
Step 4: Compute LCM = (A × B) / GCD
Step 5: Print LCM
Step 6: Stop

Example (LCM of 12 and 18)

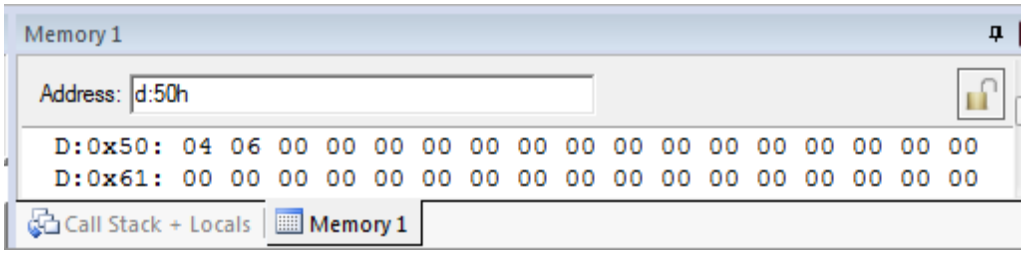
- GCD(12, 18) = 6
- LCM = (12 × 18) / 6 = 216 / 6 = **36**

CODE:

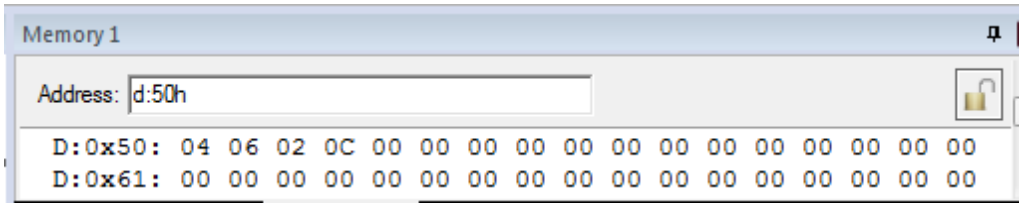
Program	Comments
a) Program to find GCD and LCM	
ORG 0000H	Starting address of Memory (Origin)
MOV A, 50H	First number is in 50H moved to register A
MOV B, 51H	Second number is in 51H moved to register B
BACK: MOV R1,B	Second number moved from B to R1 Register
DIV AB	Divide A/B (Result The quotient is stored in register A,
MOV A, B	and the remainder is stored in register B after the
CJNE A, #00H, L1	division)
L1: JZ LAST	Remainder is compared with zero
MOV A, R1	If zero go to last and terminate; otherwise repeat
SJMP BACK	
LAST: MOV A, R1	
MOV 52H, A	
MOV A, 50H	
MOV B, 51H	Store the GCD value in 52H
MUL AB	First number is in 50H moved to register A
MOV B, 52H	Second number is in 51H moved to register B
DIV AB	Multiply A and B result stored in A
MOV 53H, A	Move GCD to B register
END	Divide A/B
	Quotient will be LCM and store in 53H
	End of the program

RESULT:

Before Execution:



After Execution:



1. **Write a program to find the GCD and LCM of a given two bytes in a memory locations**

```
ORG 0000H
MOV A, 50H
MOV B, 51H
BACK: MOV R1,B
DIV AB
MOV A, B
CJNE A, #00H, L1
L1: JZ LAST
MOV A, R1
SJMP BACK
LAST: MOV A, R1
MOV 52H, A
MOV A, 50H
MOV B, 51H
MUL AB
MOV B, 52H
DIV AB
MOV 53H, A
END
```

2. **Write a program to convert the given BCD number into its equivalent seven segment value.**

```
ORG 0000H
MOV P1, #0FFH
MOV P2, #00H
MOV DPTR, #0300H
AGAIN: MOV A, P1
ANL A, #0FH
MOVC A,@A+DPTR
MOV P2, A
SJMP AGAIN
ORG 0300H
DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH
END
```

3. Toggling of p1.5 with the delay of 5ms without interrupts using timer 1

```
ORG 0000H
MOV TMOD, #10h
AGAIN: MOV TL1, #0FFH
MOV TH1, #0EDH
CPL P1.5
SETB TR1
BACK: JNB TF1, BACK
CLR TR1
CLR TF1
SJMP AGAIN
END
```

4. Toggling of p1.2 with the delay of 5ms with interrupts using timer 0

```
ORG 0 LJMP
MAIN
ORG 000BH; ISR for Timer 0
CPL P1.2
MOV TL0, #0FFH
MOV TH0, #0EDH
RETI
ORG 30H
MAIN: MOV TMOD, #01h; Timer 0, Mode 1
MOV TL0, #00
MOV TH0, #0DCH
MOV IE, #82H; enable Timer 0 interrupt
SETB TR0
HERE: SJMP HERE
END
```

5. Count no. of pulses without interrupts using counter 0

```
ORG 0000H
MOV   TMOD,  #05h
SETB P3.4
MOV   TL0,   #00H
MOV   TH0,   #00H
SETB TR0
BACK: MOV A, TL0
MOV P1, A
MOV A, TH0
MOV P2, A
SJMP BACK
END
```

6. Count no. of pulses with interrupts using counter 1

```
ORG 0000H
LJMP MAIN
ORG 001BH
MOV P1, #55H
MOV TL1, #0FAH
MOV TH1, #0FFH
RETI
ORG 30H
MAIN: MOV TMOD, #50h
SETB P3.5
MOV TL1, #0FAH
MOV TH1, #0FFH
MOV IE, #88H
SETB TR1
HERE: MOV P1, #0AAH
SJMP HERE
END
```

7. Program to transfer data serially with baud rate of 9600

```
MOV TMOD, #20H
MOV TH1, #-3
MOV SCON, #50H
SETB TR1
AGAIN: MOV SBUF, #"A"
HERE: JNB TI, HERE
CLR TI
SJMP AGAIN
END
```

8. Program to receive data serially with baud rate of 9600

```
ORG 0000h
MOV TMOD, #20H
MOV TH1, #-3 MOV
SCON, #50H
SETB TR1
HERE: JNB RI, HERE
MOV A, SBUF
MOV P1, A
CLR RI
SJMP HERE
END
```

9. Sequence Generator Using Serial Interface in 8051

```
TX_CHAR EQU 175DH
DELAY EQU 6850H
ORG 0000H
LJMP MAIN_START
ORG 9000H
MAIN_START: MOV R4, #08
MOV A, #01H
NXT_CH: MOV R6, A
MOV B, A
ADD A, #30H
CJNE A, #3AH, N00
N00: JC N05
ADD A, #07H
N05: MOV SBUF, A
LCALL UART_TX_CHAR
MOV A, B
ADD A, #02H
DJNZ R4, NXT_CH
SJMP $
ORG 175DH
UART_TX_CHAR: JNB TI, $
CLR TI
RET
END
```

Experiment # 10 To Programming for Delay Generation and Effect of CPU Clock

Aim

To study the effect of CPU clock on Microcontrollers Operations

Objective

To investigate the impact of varying the clock speed of a PIC18 series microcontroller on the speed of executing instructions, and to understand how this parameter affects the overall performance of the microcontroller

Theory

A microcontroller is a computer in miniature form that is incorporated on a single integrated circuit. It is commonly referred to as a "system on a chip". A microcontroller typically contains one or more central processing units (CPUs), memory, and programmable input/output peripherals, as well as a small amount of RAM and program memory. Unlike the microprocessors used in personal computers or other general-purpose applications that are composed of several separate chips, microcontrollers are designed specifically for embedded applications.

The PIC18 series microcontroller is a popular microcontroller family that is commonly used in various embedded systems applications due to its low power consumption and ease of programming. The GPIO (General Purpose Input Output) pins of a microcontroller are used to connect external devices such as LEDs, switches, sensors, etc. In this experiment, we will be using a push button switch to control the blinking of an LED connected to a GPIO pin of the PIC18 microcontroller.

Overall, this experiment will demonstrate the fundamental concept of clock of a microcontroller and controlling an LED's blinking. This concept can be further extended to understand how the clock is an essential parameter used by microcontroller.

A microcontroller is made of several registers. Some used for general purpose, and some are special function registers. For an 8-bit microcontroller, all actions are performed through registers having size of 8 bits.

In this experiment, we will use a special kind of SFR inside PIC18f4550 known as OSCCON.

The OSCCON register controls several aspects of the device clock's operation, both in full-power operation and in power-managed modes.

OSCCON: OSCILLATOR CONTROL REGISTER

R/W-0	R/W-1	R/W-0	R/W-0	R(1)	R-0	R/W-0	R/W-0
IDLEN	IRCF2	IRCF1	IRCF0	OSTS	IOFS	SCS1	SCS0
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7	**IDLEN:** Idle Enable bit 1 = Device enters Idle mode on SLEEP instruction 0 = Device enters Sleep mode on SLEEP instruction
bit 6-4	**IRCF2:IRCF0:** Internal Oscillator Frequency Select bits 111 = 8 MHz (INTOSC drives clock directly) 110 = 4 MHz 101 = 2 MHz 100 = 1 MHz(3) 011 = 500 kHz 010 = 250 kHz 001 = 125 kHz 000 = 31 kHz (from either INTOSC/256 or INTRC directly)(2) **IRCF2:IRCF0:** Internal Oscillator Frequency Select bits 111 = 8 MHz (INTOSC drives clock directly)
bit 3	**OSTS:** Oscillator Start 1 = Oscillator Start 0 = Oscillator Start
bit 2	**IOFS:** INTOSC Frequency Stable bit 1 = INTOSC frequency is stable 0 = INTOSC frequency is not stable
bit 1-0	**SCS1:SCS0:** System Clock Select bits 1x = Internal oscillator 01 = Timer1 oscillator 00 = Primary oscillator

Fig: Oscillator Control Register

Schematic

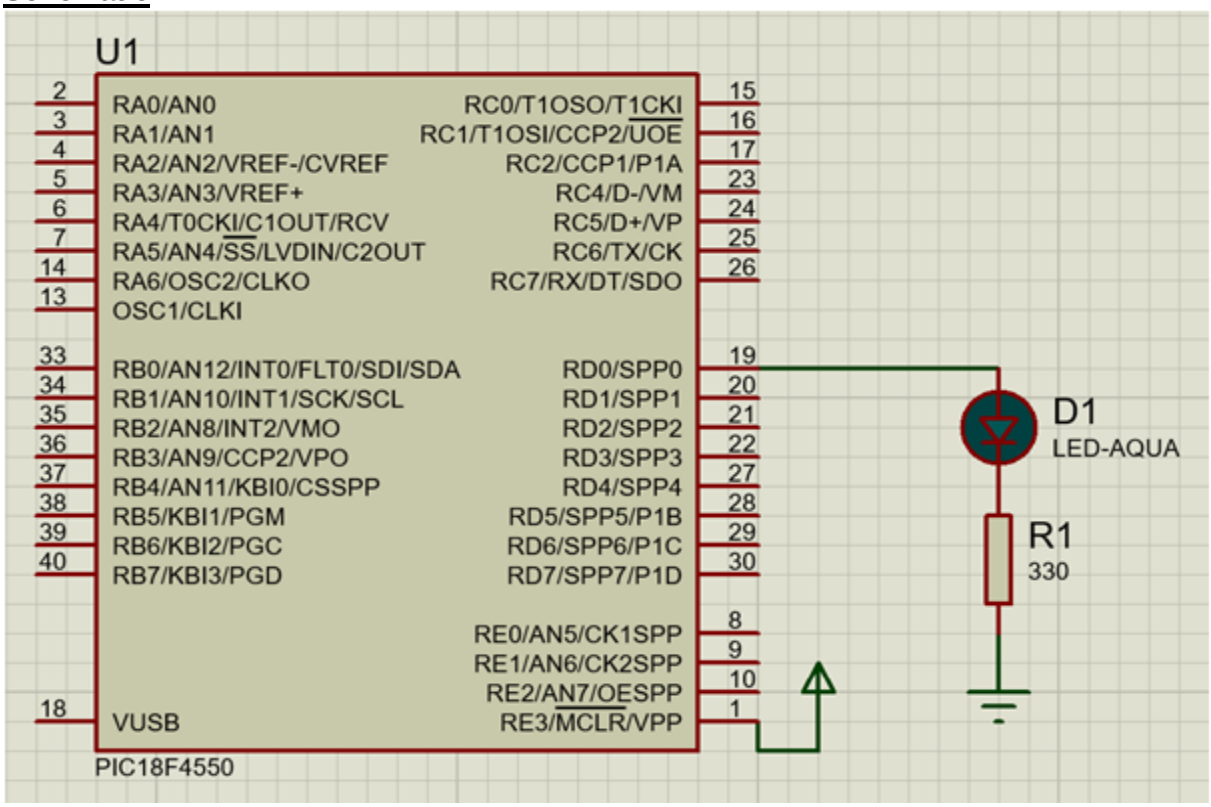


Fig: LED interfacing with microcontroller (Current sourcing)

Pre-test (Quiz)

1) What is the function of a microcontroller?

a: To process data and execute instructions

b: To provide power to the circuit
c: To control the temperature of the microcontroller
d: To store data

2) How does the CPU clock affect the operations of a microcontroller?
a: It determines the speed at which the instructions are executed
b: It determines the amount of memory available to the microcontroller
c: It determines the size of the microcontroller
d: It determines the type of instructions that can be executed

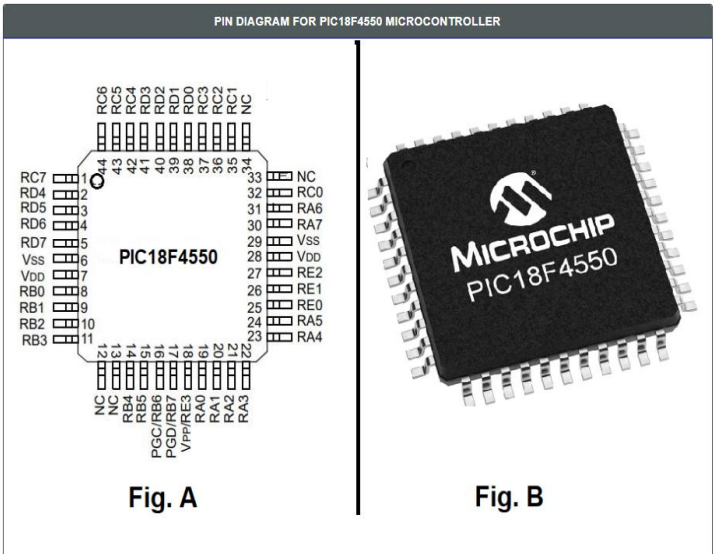
3) What is a clock cycle?
a: The time it takes for a microcontroller to execute one instruction
b: The time it takes for a microcontroller to turn on
c: The time it takes for a microcontroller to turn off
d: The time it takes for a microcontroller to switch from one task to another

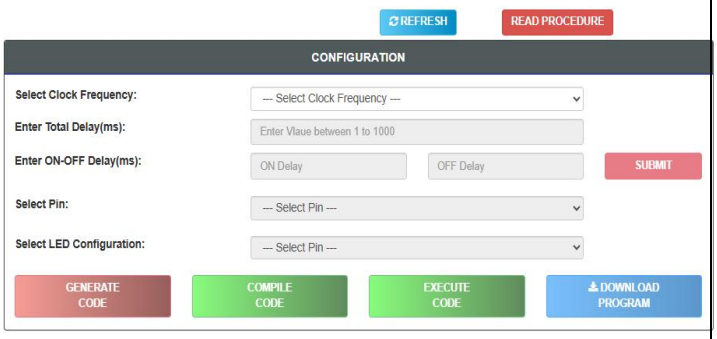
4) How does the clock speed affect the accuracy of the microcontroller's timing functions?
a: The higher the clock speed, the more accurate the timing functions
b: The lower the clock speed, the more accurate the timing functions
c: The accuracy of the timing functions is not affected by the clock speed
d: The accuracy of the timing functions is determined by the type of instructions being executed

Procedure
Simulator :

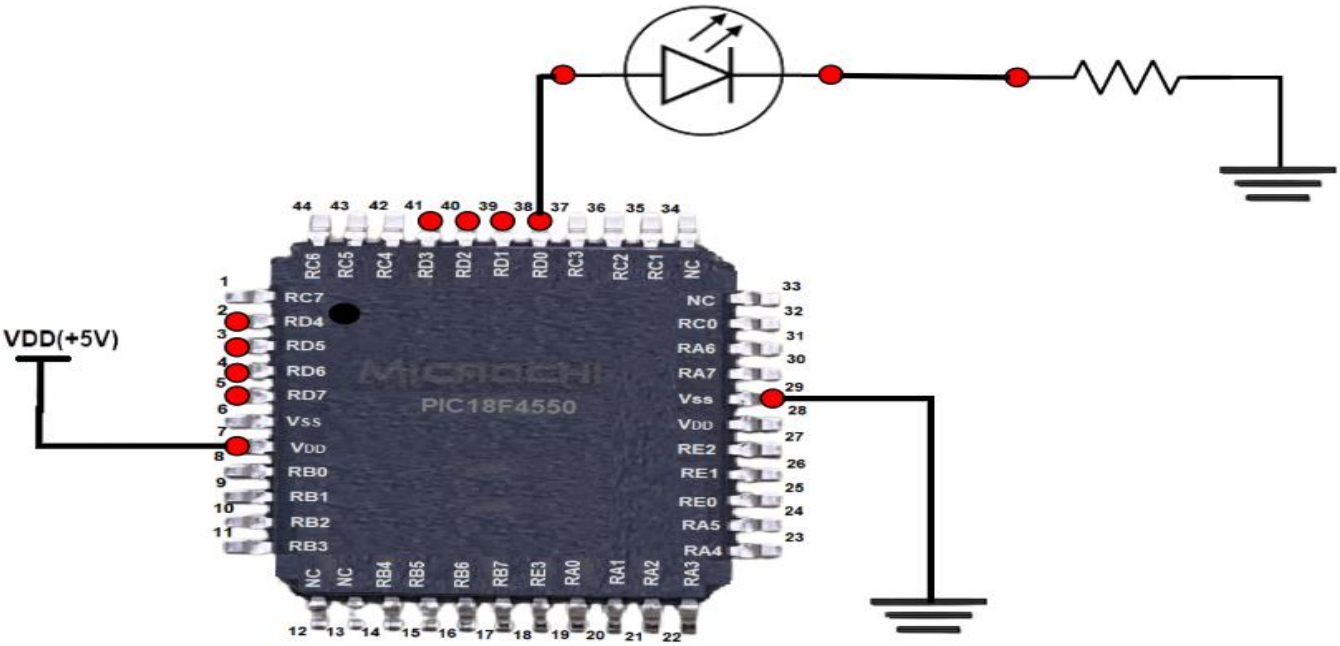
- step 1 : Select clock frequency.
- step 2 : Enter Total Delay(ms) between 1 ms to 1000 ms.
- step 3 : Enter ON Delay and OFF Delay in milliseconds. The Addition of ON Delay and OFF Delay must be same as Total Delay.
- step 4 : Click on "SUBMIT" button.
- step 5 : Select Pin.
- step 6 : Select LED Configuration.
- step 7 : Click on "GENERATE CODE" button to generate code.
- step 8 : Click on "COMPILE" button to compile code.
- step 9 : Click on "EXECUTE" button to execute code and goto mimic.
- step 10 : If you want to download code click on "DOWNLOAD" button.
- step 11 : The MIMIC will open on left side of screen.
- step 12 : Click on selected pin terminal(red circle) and click on anode terminal(red circle) for connection of the selected pin to anode or vice versa.
- step 13 : Click on cathode terminal(red circle) and click on resistor terminal(red circle) for connection of cathode to resister or vice versa.
- step 14 : Click on "Check Connection" button.
- step 15 : If connection is wrong, click on "Reset Connection" button and follow step 1, 2 and 3.
- step 16 : If connection is successful, click on "Run" button.

- step 17 : Output LED is on. Observe the square wave graph.
- step 18 : Click on "REFRESH" button to re-executing the experiment.





Check Connection | Run | Reset Connection



Source code :

```
#include #include
#pragma config XINST=OFF
#pragma config FOSC = INTOSCIO_EC
#pragma config WDT = OFF #pragma config LVP = OFF
#define LED_TRIS_Output TRISDbits.TRISD0
 #define LED_LAT_Output LATDbits.LATD0
void main(void) {
OSCCON = 0x42;
 LED_TRIS_Output=0;
 LED_LAT_Output=0;
 while(1)
{
LED_LAT_Output != LED_LAT_Output; //LED toggle
delay1ktcy(400);
}
}
```

Conclusion :

We have understood the Programming for effect of CPU Clock on Microcontrollers operations using Virtual Lab

Experiment # 11 Programming with on-chip Timers/Counters

Aim

Programming with on-chip Timers/Counters

Objective

- To understand the basic concepts of Timer and Counter
- To study in detail Timer0 of PIC Microcontroller
- To Study interrupt structure of PIC Microcontroller
- To use timer interrupt and its related SFR

Theory :

1. Timer

The microcontroller oscillator uses quartz crystal for its operation. Even though it is not the simplest solution, there are many reasons to use it. The frequency of such oscillator is precisely defined and very stable, so that pulses it generates are always of the same width, which makes them ideal for time measurement. Such oscillators are also used in quartz watches. If it is necessary to measure time between two events, it is sufficient to count up pulses generated by this oscillator. This is exactly what the timer does. Most programs use these miniature electronic 'stopwatches'. These are commonly 8- or 16-bit SFRs the contents of which are automatically incremented by each coming pulse. Once a register is completely loaded and overflowed, an interrupt may be generated! If the timer uses an internal quartz oscillator for its operation then it can be used to measure time between two events (if the register value is A at the moment measurement starts, and B at the moment it terminates, then the elapsed time is equal to the result of subtraction B - A). If registers use pulses coming from external source then such a timer is turned into a counter.

1.1 How does the timer operate?

In practice, pulses generated by the quartz oscillator are once per each machine cycle, directly or via a prescaler, brought to the circuit which increments the number stored in the timer register. If one instruction (one machine cycle) lasts for four quartz oscillator periods then this number will be incremented a million times per second (each microsecond) by embedding quartz with the frequency of 4MHz. It is easy to measure short time intervals, up to 256 microseconds, in the way described above because it is the largest number that one register can store. This restriction may be easily overcome in several ways such as by using a slower oscillator, registers with more bits, prescaler or interrupts. The first two solutions have some weaknesses so it is more recommended to use prescaler or interrupt.

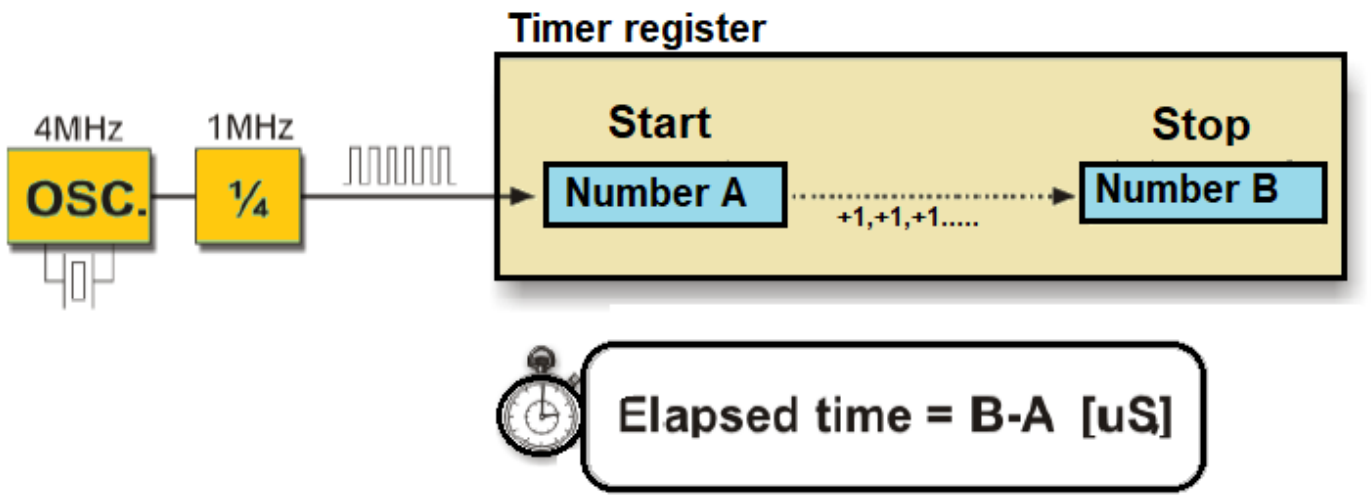


Figure 1 : Timer Operation

1.2 Using a prescaler in timer operation

A prescaler is an electronic device used to reduce frequency by a predetermined factor. In order to generate one pulse on its output, it is necessary to bring 1, 2 , 4 or more pulses on its input. Most microcontrollers have one or more prescaler built in and their division rate may be changed from within the program. The prescaler is used when it is necessary to measure longer periods of time. If one prescaler is shared by timer and watchdog timer, it cannot be used by both of them simultaneously.

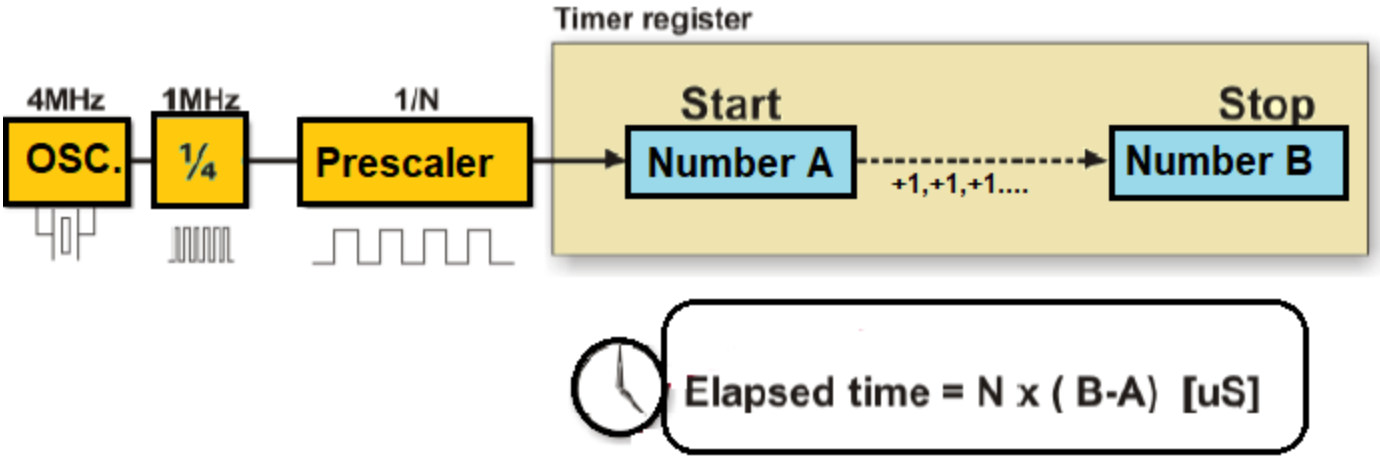


Figure 2: Use of Prescaler

1.3 Using interrupt in timer operation

If the timer register consists of 8 bits, the largest number it can store is 255. As for 16-bit registers it is the number 65535. If this number is exceeded, the timer will be automatically reset and counting will start at zero again. This condition is called an overflow. If enabled from within the program, the overflow can cause an interrupt, which gives completely new possibilities. For example, the state of registers used for counting seconds, minutes or days can be changed in an interrupt routine. The whole process (except for interrupt routine) is automatically performed behind the scenes, which enables the main circuits of the microcontroller to operate normally. This figure 3 illustrates the use of an interrupt in timer operation. Delays of arbitrary duration, having almost no influence on the main program execution, can be easily obtained by assigning the prescaler to the timer.

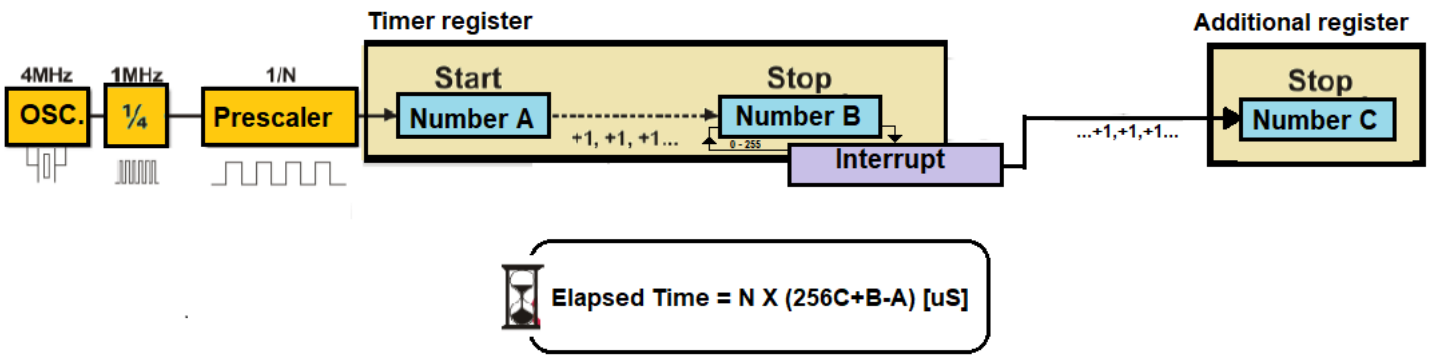


Figure 3: Timer Interrupt Operation

2. Counters

If the timer receives pulses from the microcontroller input pin, then it turns into a counter. Obviously, it is the same electronic circuit able to operate in two different modes. The only difference is that in this case pulses to be counted come over the microcontroller input pin and their duration (width) is mostly undefined. This is why they cannot be used for time measurement, but for other purposes such as counting products on an assembly line, number of axis rotation, passengers etc. (depending on sensor in use).

3. Timers / Counters in PIC Microcontroller

There are 3 - 5 timers on board in PIC microcontroller. These timers can also be used as counters when external pulses are applied. The timers are programmable, and sometimes share with other peripheral devices. These are named as TMR0, TMR1, TMR2, TMR3 and TMR4. There are few other timers, not to be discussed here like Watchdog Timer and Brown-Out timers. These timers are useful in measuring the time delays in various events as well as counting and timing external events.

Comparison of Timers in PIC Microcontroller

Parameter	Timer 0	Timer 1 & 3	Timer 2 & 4
Size of timer register	8-bit or 16-bit	16-bit	8-bit
Clock Source (Internal)	Fosc/4	Fosc/4	Fosc/4
Clock Source (External)	T0CKI Pin	T13CKI Pin/T1OSC	None
Clock Scaling (Prescaler)	Prescaler 8-bits(1:2 -> 1:256)	Prescaler 2-bits (1:1, -> 1:8)	prescaler (1:1, 1:4, 1:8) Postscaler(1:1 ->1:16)
Interrupt Event	On Overflow	On Overflow	TMR Reg matches with PR2

4. Timer0 Module in PIC Microcontroller

The Timer0 module incorporates the following features:

- Software selectable operation as a timer or
- counter in both 8-bit or 16-bit modes
- Readable and writable registers
- Dedicated 8-bit, software programmable
- prescaler
- Selectable clock source (internal or external)
- Edge select for external clock
- Interrupt on overflow

The T0CON register (Register 11-1) controls all aspects of the module's operation, including the prescale selection. It is both readable and writable. Figure A. shows a simplified block diagram of the Timer0 module in 8-bit mode. Figure B. shows a simplified block diagram of the Timer0 module in 16-bit mode.

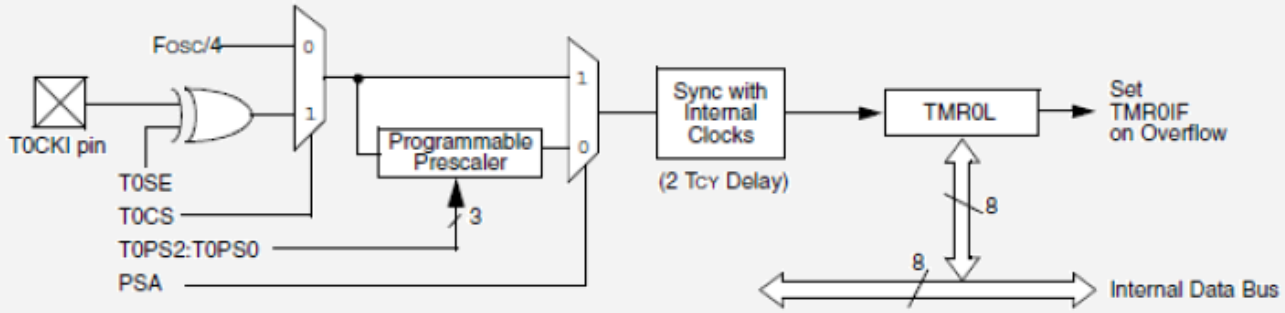


Figure A : Block Diagram of Time0 in 8-bit

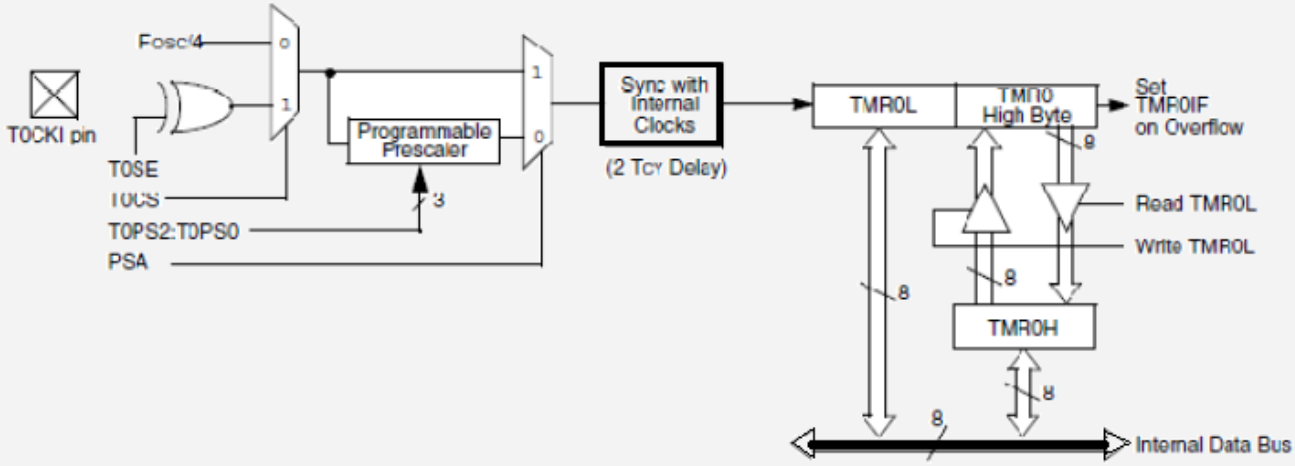


Figure B : Block Diagram of Time0 in 16-bit

4.1 Timer0 Operation

Timer0 can operate as either a timer or a counter; the mode is selected by clearing the T0CS bit (T0CON<5>). In Timer mode, the module increments on every clock by default unless a different prescaler value is selected. If the TMR0 register is written to, the increment is inhibited for the following two instruction cycles. The user can work around this by writing an adjusted value to the TMR0 register. The Counter mode is selected by setting the T0CS bit (= 1). In Counter mode, Timer0 increments either on every rising or falling edge of pin RA4/T0CKI. The incrementing edge is determined by the Timer0 Source Edge Select bit, T0SE (T0CON<4>); clearing this bit selects the rising edge. An external clock source can be used to drive Timer0; however, it must meet certain requirements to ensure that the external clock can be synchronized with the internal phase clock (TOSC). There is a delay between synchronization and the onset of incrementing the timer/counter.

4.2 Prescaler

An 8-bit counter is available as a prescaler for the Timer0 module. The prescaler is not directly readable or writable; its value is set by the PSA and T0PS2:T0PS0 bits (T0CON<3:0>) which determine the prescaler assignment and prescale ratio. Clearing the PSA bit assigns the prescaler to the Timer0 module. When it is assigned, prescale values from 1:2 through 1:256, in power-of-2 increments are selectable. When assigned to the Timer0 module, all instructions writing to the TMR0 register clear the prescaler count.

4.3 Timer0 Interrupt

The TMR0 interrupt is generated when the TMR0 register overflows from FFh to 00h in 8-bit mode, or from FFFFh to 0000h in 16-bit mode. This overflow sets the TMR0IF flag bit. The interrupt can be masked by clearing the TMR0IE bit (INTCON<5>). Before reenabling the interrupt, the TMR0IF bit must be cleared in software by the Interrupt Service Routine. Since Timer0 is shut down in Sleep mode, the TMR0 interrupt cannot awaken the processor from Sleep.

4.4 Timer0 Register Map:

SFR	Description	Access	Reset Value	Address
T0CON	Timer0 Control Register	Read/Write	0xFF	0xFD5

SFR	Description	Access	Reset Value	Address
TMR0L	Timer0 Register Lower Byte	Read/Write	UNKNOWN	0xFD6
TMR0H	Timer0 Register Higher Byte	Read/Write	0x00	0xFD7
INTCON	Interrupt Control Register	Read/Write	0x00	0xFF2
INTCON2	Interrupt Control Register 2	Read/Write	0xFF	0xFF1

Register (SFR) Description

4.4.1 TMR0L :

Used 8-bit and 16bit mode. The register holds the current count value which is updated by clock source. User must write initial value.

4.4.2 TMR0H :

Used only in 16-bit mode. The register holds the higher byte current count value which is updated by clock source.

4.4.3 T0CON: Timer0 Control Register

R/W - 1	R/W - 1	R/W - 1	R/W - 1	R/W - 1	R/W - 1	R/W - 1	R/W - 1
TMR0ON	T08BIT	T0CS	T0SE	PSA	T0PS2	T0PS1	T0PS0
bit-7							bit-0

Bit No.	Control Bit	Description
Bit 7	TMR0ON	Timer0 On/Off Control bit 1 = Enables Timer0, 0 = Stops Timer0
Bit 6	T08BIT	Timer0 8-Bit/16-Bit Control bit 1 = Timer0 is configured as an 8-bit timer/counter 0 = Timer0 is configured as a 16-bit timer/counter
Bit 5	T0CS	Timer0 Clock Source Select bit 1= Transition on T0CKI pin 0 = Internal instruction cycle clock (CLKO)
Bit 4	T0SE	Timer0 Source Edge Select bit 1 = Increment on high-to-low transition on T0CKI pin 0 = Increment on low-to-high transition on T0CKI pin
Bit 3	PSA	Timer0 Prescaler Assignment bit 1 = TImer0 prescaler is NOT assigned. 0 = Timer0 prescaler is assigned.
Bit 2-0	T0PS2:T0PS0	Timer0 Prescaler Select bits 111 = 1:256 Prescale value 110 = 1:128 Prescale value 101 = 1:64 Prescale value 100 = 1:32 Prescale value 011 = 1:16 Prescale value 010 = 1:8 Prescale value 001 = 1:4 Prescale value 000 = 1:2 Prescale value

4.4.4 INTCON: Interrupt Control Register

R/W - 0	R/W - 0	R/W - 0	R/W - 0	R/W - 0	R/W - 0	R/W -0	R/W - 0
GIE/GIEH	PEIE/GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF^1
bit-7							bit-0

Bit No.	Control Bit	Description
Bit 7	GIE/GIEH	Global Interrupt Enable bit When IPEN = 0: 1 = Enables all unmasked interrupts 0 = Disables all interrupts When IPEN = 1: 1 = Enables all high priority interrupts 0 = Disables all high priority interrupts
Bit 6	PEIE/GIEL	Peripheral Interrupt Enable bit
Bit 5	TMR0IE	TMR0 Overflow Interrupt Enable bit 1 = Enables the TMR0 overflow interrupt 0 = Disables the TMR0 overflow interrupt
Bit 4	INT0IE	INT0 External Interrupt Enable bit 1 = Enables the INT0 external interrupt 0 = Disables the INT0 external interrupt
Bit 3	RBIE	RB Port Change Interrupt Enable bit 1 = Enables the RB port change interrupt 0 = Disables the RB port change interrupt
Bit 2	TMR0IF	TMR0 Overflow Interrupt Flag bit 1 = TMR0 register has overflowed (must be cleared in software) 0 = TMR0 register did not overflow
Bit 1	INT0IF	INT0 External Interrupt Flag bit 1 = The INT0 external interrupt occurred (must be cleared in software) 0 = The INT0 external interrupt did not occur
Bit 0	RBIF	RB Port Change Interrupt Flag bit(1) 1 = At least one of the RB7:RB4 pins changed state (must be cleared in software) 0 = None of the RB7:RB4 pins have changed state

5. Time delay generation using timer:
The following steps are taken to generate a time delay using polling method
i. Load the proper value in T0CON indicating which timer mode, clock source, prescaler assignment.
ii. Load the registers TMR0H first and then TMR0L with initial count values. Delay generated is Depends upon the initial count value.
iii. Start the timer by setting TMR0ON bit in T0CON.
iv. Keep the monitoring the timer flag (TMR0IF).
v. Get out of the loop when TMR0IF becomes high.
vi. Stop the timer.
vii. Clear the TMR0IF flag for the next round.
viii. Go back to the step 2 to load TMR0H and TMR0L values.

The following steps are taken to generate a time delay using interrupt method
i. Load the proper value in T0CON indicating which timer mode, clock source, prescaler assignment.

ii. Load the registers TMR0H first and then TMR0L with initial count values. Delay generated is Depends upon the initial count value.
iii. Enable the Timer0 Interrupt and Global Interrupt using INTCON Register
iv. Start the timer by setting TMR0ON bit in T0CON.
v. Write the ISR at Interrupt vector 0x0018.
a. In ISR Clear the TMR0IF flag for the next round.
b. In ISR reload TMR0H and TMR0L values. The size of the time delay depends on two factors, (a) the crystal frequency and (b) the timer's 16-bit register (c) the prescaler value. The largest delay is achieved by the making both TMR0H and TMR0L zero and using maximum prescaler value i.e. 1:256.

Formula for delay calculations using timer for crystal frequency of F_{osc} = 20 MHz, (TMR0H, TMR0L) = $(NNNNN)_{10}$ is as follows.

Clock applied to timer0 = Timer Clock Freq. = $\mathbf{F_T} = F_{osc}/4 = 20x10^6/4 = 5$ MHz

Therefore Time Period $\mathbf{T_F} = 1/5x10^6 = 0.2$

(a) Without Prescaler

Time Delay (Td) = (65536 - NNNNN) x Time Period $\mathbf{T_F}$

Therefore Maximum delay = (65536 – 00000) x 0.2 us = 13.10 ms

(a) With Prescaler (1:256)

Time Delay (Td) = (65536 - NNNNN) x (Timer Period $\mathbf{T_F}$ x Prescaler Value)

Therefore Maximum delay = (65536 – 00000) x (0.2 us x 256) = 3.36 seconds

5.1 Finding the Values to be loaded into timer for desired delay

(a) Without Prescaler

- Divide the desired time delay **Td** by $\mathbf{T_F}$ to get n(n = Td/0.2 us for Fosc = 20MHz)
- Perform 65536 – n. where n is the decimal value from step 1.
- Convert the result of step 2 to hexadecimal, where yyxx is the initial hex value to be loaded into the timer's registers.
- Set TMR0L = xx and TMR0H = yy.

(b) With Prescaler

- Multiply Timer period $\mathbf{T_F}$ by prescaler value to Teq

  $\mathbf{T_{eq} = T_F * Prescaler\ vale}$
- Divide the desired time delay **Td** by $\mathbf{T_{eq}}$ to get n

  n = Td/Teq
- Perform 65536 – n. where n is the decimal value from above.
- Convert the result of above step to hexadecimal, where yyxx is the initial hex value to be loaded into the timer's registers.
- Set TMR0L = xx and TMR0H = yy.

5.2 Calculation for generating square wave of 10Hz.

- Desired frequency of Square Wave Fs = 10 Hz
- Ts = 1/Fs = 1/10 = 0.1 second
- On Period and OFF period of square wave = Ts/2 = 0.05s = 50 ms
- To generate square wave the value of port pin RB0 should be toggled after every 50 ms.
- From above calculation we have to use prescaler (1:4 or 1:8) to generate delay of 50 ms since maximum delay generated by timer is 13.10 ms without using prescaler for Fosc = 20Mhz.

5.2.1 For Operating Frequency Fosc = 20MHz.

- Selecting Prescaler 1:8, Timer Clock source = Fosc/4
- Therefore Timer Clock Freq. = $\mathbf{F_T} = F_{osc}/4 = 20x10^6/4 = 5$ MHz
- $T_{eq} = T_F$ * Prescaler vale = 0.2 us x 8 = 1.6 us
- To get n, n= Td/Teq = 50ms/1.6us = 31250
- $(yyxx)_{10}$ = 65536 – n = 65536 – 31250 = 34286
- $(34286)_{10} = (85EE)_{16}$ should be loaded in TMR0H:TMR0L.

5.2.2 For Operating Frequency Fosc = 48MHz.

- Selecting Prescaler 1:16, Timer Clock source = Fosc/4
- Therefore Timer Clock Freq. = $\mathbf{F_T} = F_{osc}/4 = 48x10^6/4 = 12$ MHz
- $T_{eq} = T_F$ * Prescaler vale = 0.08333 us x 16 = 1.333 us
- To get n, n= Td/Teq = 50ms/1.333 us = 37501
- $(yyxx)_{10}$ = 65536 – n = 65536 – 37501 = 28034
- $(28034)_{10} = (6D82)_{16}$ should be loaded in TMR0H:TMR0L.

Pre-Test (Quiz)

1) How many timer are available in PIC18F4550?
a: 1
b: 2
c: 3
d: 4

2) Which timer is used for PWM generation?
a: Timer0
b: Timer1
c: Timer2
d: Timer3

3) What is the primary purpose of a timer in a microcontroller?
a: To store data permanently
b: To count external events like button presses
c: To measure time intervals using clock pulses
d: To increase the microcontroller's clock speed

4) Which of the following is NOT a way to extend the timer's maximum measurable time?
a: Using a prescaler
b: Using interrupts
c: Using a slower oscillator
d: Increasing the clock speed

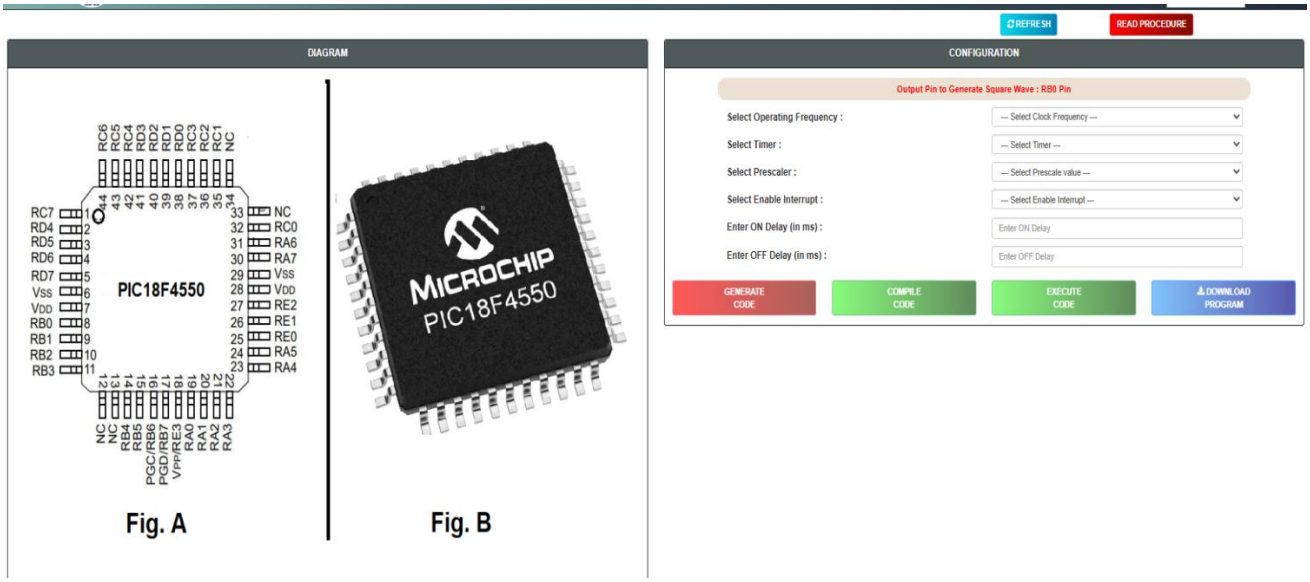
5)What is the function of a prescaler in a timer operation?
a: To increase the timer's resolution
b: To reduce the frequency of clock pulses before feeding them to the timer
c: To reset the timer on overflow
d: To enable the timer's interrupt feature

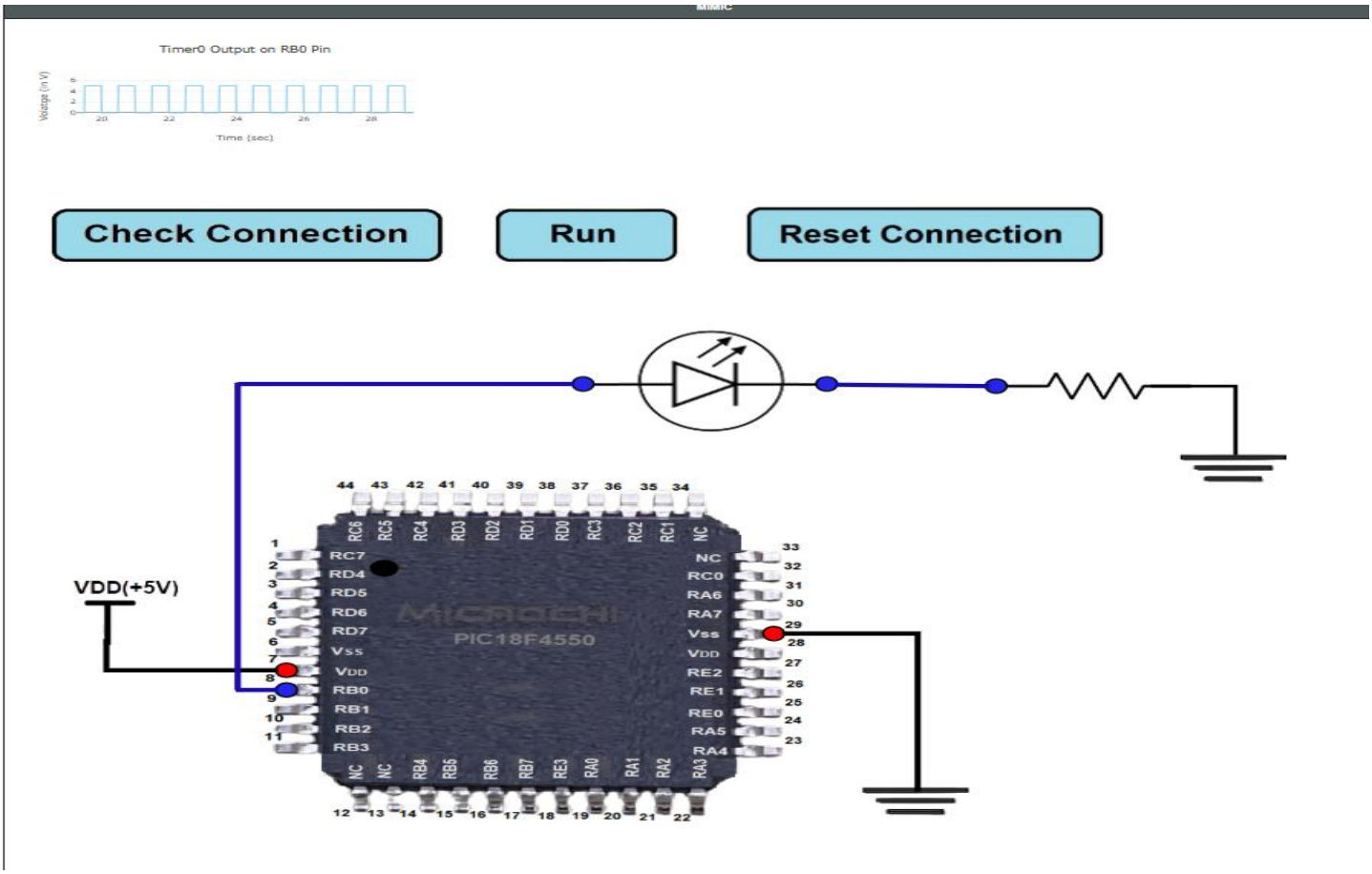
6)Which timer(s) in a PIC microcontroller has both prescaler and postscaler?
a: Timer 0
b: Timer 1
c: Timer 2 & 4
d: None of the above

Procedure
Simulator:

- step 1 : Select Operating Frequency.
- step 2 : Select Timer.
- step 3 : Select Prescaler.
- step 4 : Select Enable Interrupt.
- step 5 : Enter ON Delay (in mS).
- step 5 : Enter OFF Delay (in mS).
- step 5 : Click on "GENERATE CODE" button to generate code.
- step 6 : Click on "COMPILE" button to compile code.
- step 7 : Click on "EXECUTE" button to execute code and goto mimic.
- step 8 : If you want to download code click on "DOWNLOAD" button.

- step 9 : The MIMIC will open on left side of screen.
- step 10 : Click on selected output pin terminal(blue circle) and click on anode terminal(blue circle) for connection of the selected output pin with anode or vice versa.
- step 11 : Click on cathode terminal(blue circle) and click on resistor terminal(blue circle) for connection of cathode with resister or vice versa.
- step 12 : Click on "Check Connection" button.
- step 13 : If connection is wrong, click on "Reset Connection" button and follow step 10, 11 and 12.
- step 14 : If connection is successful, click on Run Button.
- step 15 : Output LED is on. Observe the square wave graph.
- step 16 : Click on "REFRESH" button to re-executing the experiment.





Source code :

```
#include<pic18fregs.h>
#include<delay.h>
#pragma config FOSC = HS //Oscillator Selection
#pragma config WDT = OFF //Disable Watchdog timer
#pragma config LVP = OFF //Disable Low Voltage Programming
#pragma config PBADEN = OFF //Disable PORTB Analog inputs
// Timer ISR Definition
#pragma interrupt timer_isr
void timer_isr(void)
{
  TMR0H = 0XF0;   //Reload the timer values after overflow
  TMR0L = 0XBE;
   PORTBbits.RB0 = ~PORTBbits.RB0;   //Toggle the RB0
   INTCONbits.TMR0IF = 0;   //Reset the timer overflow interrupt flag
}
//Interrupt Vector for Timer0 Interrupt
#pragma code _HIGH_INTERRUPT_VECTOR = 0x0008
void high_ISR (void)
{
   _asm
   goto timer_isr   //Call Timer ISR Routine
   _endasm
}
#pragma code
// Start of main Program
void main()
{
```

```
  ADCON1 = 0x0F;   //Configuring the PORTB pins as digital I/O
  TRISBbits.TRISB0 = 0;   //Configuring the RB0 as output
  PORTBbits.RB0 = 0;   //Setting the initial value
  T0CON = 0x07;   //Set the timer to 256-bit,Fosc/4,1:256 prescaler
  TMR0H = 0XF0;   //load timer value to generate delay 50ms
 TMR0L = 0XBE;
  INTCONbits.TMR0IF = 0;   // Clear Timer0 overflow flag
  INTCONbits.TMR0IE = 1;   // TMR0 interrupt enabled
  T0CONbits.TMR0ON = 1;   // Start timer0
  INTCONbits.GIE = 1;   // Global interrupt enabled
  while(1);   //Program execution stays here until the
    //timer overflow interrupt is generated
}
```

Conclusion :

We have understood the Programming for On-Chip Timers /counters using Virtual Lab.

Experiment #12 Programming with On-Chip ADC

Aim

Programming with on-chip ADC

Objective

1. To understand the working of A/D converter.
2. To study the on-chip ADC section of PIC Microcontroller.
3. To interface analog input to PIC Microcontroller.

Theory :

1. ADC Devices :

Analog to digital converters are among the most widely used devices for data acquisitions. Digital computers use binary (discrete) value but in physical world everything is analog (continuous). A physical quantity is converted to electrical signals using device called transducer or also called as sensors. Sensors and many other natural quantities produce an output that is voltage (or current). Therefore we need an analog - to - digital converter to translate the analog signal to digital numbers so that the microcontroller can read and process them. The ADC chips are either parallel or serial. In parallel ADC, we have 8 or more pins dedicated to bring out the binary data, but in serial ADC we have only one pin for data out. Some of the major characteristics of ADC are :

a) resolution
b) conversion time
c) reference voltage (Vref).

1.1 Resolution

An ADC has an n bit resolution where n can be 8, 10, 16, 0r even 24 bits. The higher resolution ADC provides a smaller step size, where step size is smallest change that can be discerned by an ADC. This is shown in below Table Although the resolution for ADC chip is designed at the time of its design and cannot be changed, we can control the step size with the help of reference voltage (Vref).

n - bit	Number of steps	Step Size (mV)
8	256	5/256 = 19.53
10	1024	5/1024 = 4.88
12	4096	5/4096 = 1.2
16	65536	5/65536 = 0.076

1.2 Conversion time

Conversion time is defined as the time it takes the ADC to convert the analog input to digital (binary) number. The conversion time is dictated by the clock source connected to the ADC in addition to the method used for data conversion and technology used in the fabrication of the ADC chip.

In addition to conversion time, acquisition time is another major factor in judging an ADC. Acquisition time is defined as the time it takes to sample the analog voltage using sample and hold circuit. Sampled analog input is applied to actual conversion unit of ADC. In some of the ADCs, conversion time doesn't contain the acquisition time but specified separately. Therefore effective conversion time is addition of specified conversion time and specified acquisition time.

1.3 Reference Voltage (Vref)

Vref is an input voltae used for the reference voltage. The voltage connected to this pin, along with resolution of the ADC chip, gives us the step size. step size = Vref / 2n where n= no. of bits For example, If the analog input range needs to be 0 to 3 Volts, Vref is connected to 3 Volts. That gives, 3V/1024 = 2.92 mV step size for 10-bit ADC.

Vref (V)	Vin (V)	Step Size (mV)

Vref (V)	Vin (V)	Step Size (mV)
5.00	0 to 5	5/1024 = 4.88
2.56	0 to 2.56	2.56/1024 = 2.5
1.024	0 to 1.024	1.024/1024 = 1

2. 10-Bit Analog-To-Digital Converter (A/D) Module of PIC uC

The Analog-to-Digital (A/D) converter module has 10 inputs for the 28-pin devices and 13 for the 40/44-pin devices. This module allows conversion of an analog input signal to a corresponding 10-bit digital number.

2.2 Pin Description:

Signal	Pin No.	Symbol
Channel 0 (AN0)	2	RA0/AN0
Channel 1 (AN1)	3	RA1/AN1
Channel 2 (AN2)	4	RA2/AN2/Vref-/CVref
Channel 3 (AN3)	5	RA3/AN3/Vref+
Channel 4 (AN4)	7	RA5/AN4/SS/HLVDIN/C2OUT
Channel 5 (AN5)	8	RE0/AN5/CK1SPP
Channel 6 (AN6)	9	RE1/AN6/CK2SPP
Channel 7 (AN7)	10	RE2/AN7/OESPP
Channel 8 (AN8)	35	RB2/AN8/INT2/VMO
Channel 9 (AN9)	36	RB3/AN9/VPO
Channel 10 (AN10)	34	RB1/AN10/INT1/SCK/SCL
Channel 11 (AN11)	37	RB4/AN11/KBIO/CSSPP
Channel 12 (AN12)	33	RB0/AN12/INT0/FLT0/SDI/SDA

2.3 ADC Register Map:

SFR	Description	Access	Reset Value	Address
ADCON0	A/D Control Register 0	Read/Write	0x00	0xFC2
ADCON1	A/D Control Register 1	Read/Write	0x00	0xFC1
ADCON2	A/D Control Register 2	Read/Write	0x00	0xFC0
ADRESH	A/D Result High Register	Read	unknown	0xFC4
ADRESL	A/D Result Low Register	Read	unknown	0xFC3

2.4 ADC Register Description

2.4.1 A/D Control Register 0 (ADCON0)

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
-	-	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON
bit 7							bit 0

Bit No.	Control Bit	Description
Bit 7 - 6	Unimplemented	Read as '0'
Bit 5 - 2	**CHS3:CHS0**	**Analog Channel Select bits** 0000 = Channel 0 (AN0) — **0001 = Channel 1 (AN1)** 0010 = Channel 2 (AN2) — 0011 = Channel 3 (AN3) 0100 = Channel 4 (AN4) — 0101 = Channel 5 (AN5) 0110 = Channel 6 (AN6) — 0111 = Channel 7 (AN7) 1000 = Channel 8 (AN8) — 1001 = Channel 9 (AN9) 1010 = Channel 10 (AN10) — 1011 = Channel 11 (AN11) 1100 = Channel 12 (AN12) — 1101 = Unimplemented 1110 = Unimplemented — 1111 = Unimplemented
Bit 1	**GO/DONE**	**A/D Conversion Status bit** 1 = A/D conversion in progress; 0 = A/D Idle
Bit 0	**ADON**	**A/D On bit** **1 = A/D converter module is enabled** 0 = A/D converter module is disabled

ADCON0 Register in our Program
bit7-6 This two bits are Unimplemented so we write this bit as 00
bit5-2 Analog input is connected to Channel 1 (AN1). so we have to write CHS3:CHS0 analog channal select bits as 0001
bit1 Start Conversion by setting this bit. Later on poll the same conversion status. Bit0 To turn ON ADC set this bit 1**.**

2.4.2 A/D Control Register 1 (ADCON1)

U-0	U-0	R/W-0	R/W-0	R/W-0(1)	R/W(1)	R/W(1)	R/W(1)
—	—	VCFG0	VCFG0	PCFG3	PCFG2	PCFG1	PCFG0
bit 7							bit 0

Bit No.	Control Bit	Description
Bit 7 - 6	Unimplemented	Read as '0'
Bit 5	**VCFG0**	Voltage Reference Configuration bit (VREF- source) 1 = VREF- (AN2); **0 = VSS**
Bit 4	**VCFG0**	Voltage Reference Configuration bit (VREF+ source) 1 = VREF+ (AN3); **0 = VDD**
bit 3-0	**PCFG3:PCFG0**	A/D Port Configuration Control bits shown in table 8.3 **1101 : Analog channel AN1**

PCFG3: PCFG0	AN12	AN11	AN10	AN9	AN8	AN7(2)	AN6(2)	AN5(2)	AN4	AN3	AN2	AN1	AN0
0000(1)	A	A	A	A	A	A	A	A	A	A	A	A	A
0001	A	A	A	A	A	A	A	A	A	A	A	A	A
0010	A	A	A	A	A	A	A	A	A	A	A	A	A
0011	D	A	A	A	A	A	A	A	A	A	A	A	A
0100	D	D	A	A	A	A	A	A	A	A	A	A	A
0101	D	D	D	A	A	A	A	A	A	A	A	A	A
0110	D	D	D	D	A	A	A	A	A	A	A	A	A
0111(1)	D	D	D	D	D	A	A	A	A	A	A	A	A
1000	D	D	D	D	D	D	A	A	A	A	A	A	A
1001	D	D	D	D	D	D	D	A	A	A	A	A	A
1010	D	D	D	D	D	D	D	D	A	A	A	A	A
1011	D	D	D	D	D	D	D	D	D	A	A	A	A
1100	D	D	D	D	D	D	D	D	D	D	A	A	A
1101	D	D	D	D	D	D	D	D	D	D	D	A	A
1110	D	D	D	D	D	D	D	D	D	D	D	D	A
1111	D	D	D	D	D	D	D	D	D	D	D	D	D

A = Analog input D = Digital I/O

Table : PCFG3:PCFG0: A/D Port Configuration Control bits:

ADCON1 Register in our Program

- Bit7-6 This two bits are Unimplemented so we write this bit as 00
- Bit 5 In our application (VREF-) supply as 0 volt so we write this bit as 0.
- Bit 4 In our application (VREF+) supply as 5 volt so we write this bit as 0.
- Bit3-0 Here we are using only one analog channal AN1. Therefor we write this bit as PCFG3:PCFG0 - 1101.

2.4.3 A/D Control Register 2 (ADCON2)

R/W-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
ADFM	**-**	**ACQT2**	**ACQT1**	**ACQT0**	**ADCS2**	**ADCS1**	**ADCS0**
bit 7							**bit 0**

Bit No.	Control Bit	Description
Bit 7	**ADFM**	**A/D Result Format Select bit** **1 = Right justified;** 0 = Left justified
Bit 6	**Un**	**Unimplemented** Read as '0'
Bit 5–3	**ACQT2:ACQT0**	**A/D Acquisition Time Select bits** 000=0 TAD, **001=2 TAD**, 010=4 TAD, 011 =6 TAD 100=8 TAD, 101=12 TAD, 110=16 TAD, 111 =20 TAD
bit 2–0	**ADCS2:ADCS0**	**A/D Conversion Clock Select bits** 000=FOSC/2, 001=FOSC/8, 010=FOSC/32, 011=FRC* 100=FOSC/4, 101=FOSC/16, **110=FOSC/64**, 111=FRC* FRC* - clock derived from A/D RC oscillator

2.4.3.1 A/D Result Format Select bit (ADFM):
We need right justified data. Therefore ADFM = '1'. If you write 1 then result is right justified as shown below;

15	14	13	12	11	10	9	8	
—	—	—	—	—	—			ADRESH
7	6	5	4	3	2	1	0	
								ADRESL

If you write 0 then result is left justified as shown below;

15	14	13	12	11	10	9	8	
								ADRESH
7	6	5	4	3	2	1	0	
		—	—	—	—	—	—	ADRESL

2.4.3.2 A/D Conversion Clock Select bits (ADCS2:ADCS0)
The A/D conversion time per bit is defined as TAD. The A/D conversion requires 11 TAD per 10-bit conversion. The source of the A/D conversion clock is software selectable. There are seven possible options for TAD as shown in table For correct A/D conversions, the A/D conversion clock (TAD) must be as short as possible but greater than the minimum TAD. Table shows the resultant TAD times derived from the device operating frequencies and the A/D clock source selected. Minimum TAD = 0.7 us (to be maintained as per datasheet) Therefore for 10bit ADC Typical conversion time = 11 TAD = 11 * 0.7 us = 7.7 us

AD Clock Source (TAD)		Maximum Device Frequency
Operation	**ADCS2:ADCS0**	**PIC18FXXXX**
2 TOSC	000	2.86 MHz
4 TOSC	100	5.71 MHz
8 TOSC	001	11.43 MHz
16 TOSC	101	22.86 MHz
32 TOSC	010	45.71 MHz
64 TOSC	110	48.0 MHz
RC**(3)**	x11	1.00 MHz

Table TAD vs. DEVICE OPERATING FREQUENCIES

2.4.3.3 A/D Acquisition Time Select bits
Figure 1 shows the acquisition time (TACQ) is depends on amplifier settling time, capacitor charging time and

temperature coefficient. TACQ = TAMP + TC + TCOFF
Where,
TAMP : Amplifier Settling Time = 0.2 us (Typical value)
TC : Holding Capacitor Chargi
= -(CHOLD)(RIC + RSS + RS) ln(1/2048) µs
= -(25 pF) (1 kΩ + 2 kΩ + 2.5 kΩ) ln(0.0004883) µs (Typical value)
= 1.05 µs
TCOFF : Temperature Coefficient = (Temp – 25°C)(0.02 µs/°C)
= (85°C – 25°C)(0.02 µs/°C) (Typical temp)
= 1.2 µs

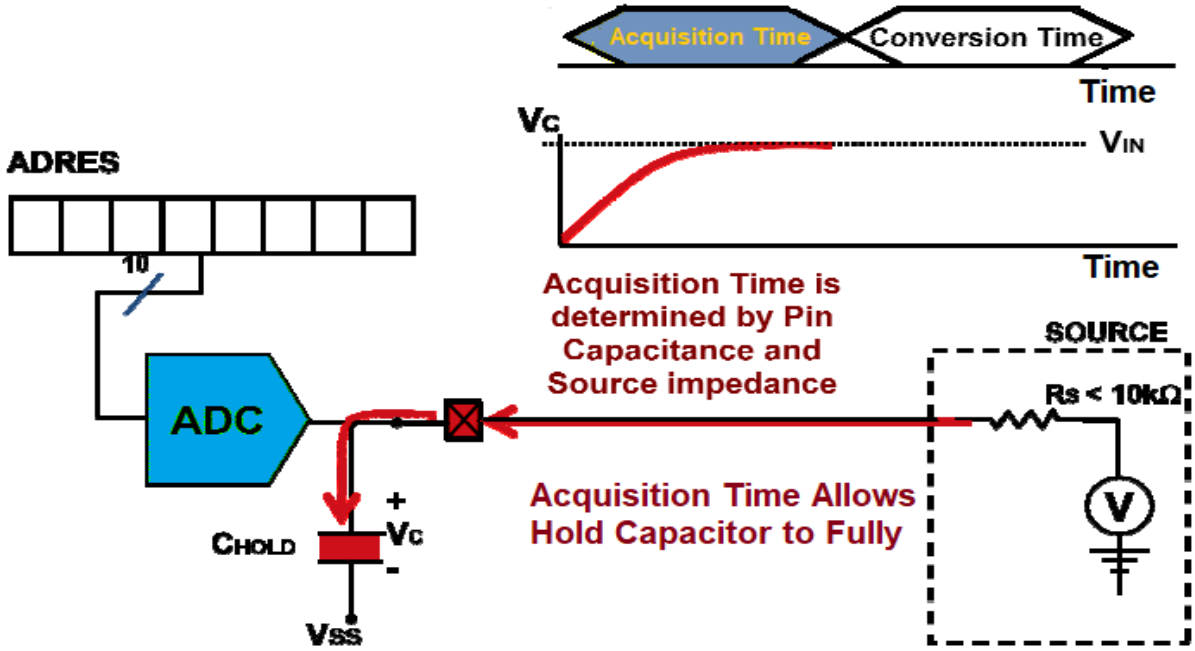


Figure 1 : Acquisition time of ADC module

Therefore minimum calculated acquisition time is
TACQ = 0.2 µs + 1.05 µs + 1.2 µs = 2.45 µs Derived A/D acquisition time ≥ Calculated acquisition time TACQ
Derived A/D acquisition time = n x TAD where n is depends on A/D Acquisition Time Select bits (ACQT2:ACQT0). So select A/D Acquisition Time Select bits (ACQT2:ACQT0) such that the we meet above timings.

2.5 Calculations

a. For Fosc = 20MHz

Tosc = 1/20MHz = 50 ns
Selected ADC clock source **ADCS2:ADCS0 = 110 i.e Fosc/64**

TAD = 64Tosc = 64 * 50 ns
TAD = 3.2 us which is much greater than minimum TAD = 0.7 us

Conversion Time = 11TAD = 11 * 3.2 us
= **35.2 us**

Selected A/D Acquisition Time Select bits **ACQT2:ACQT0 = 001 i.e n= 2**
Derived A/D acquisition time = n * TAD = 2 * TAD
= 2 * 3.2 us
= 6.4 us
Which is much greater than **calculated acquisition time = 2.4 us**

Effective conversion time = acquisition time + conversion time
= 35.2 us + 6.4 us
= 41.6 us

b. For Fosc = 48MHz

Tosc = 1/48MHz = 20.83 ns
Selected ADC clock source **ADCS2:ADCS0 = 110 i.e Fosc/64**

TAD = 64Tosc = 64 * 20.83 ns
TAD = 1.33 us which is greater than minimum TAD = 0.7 us

Conversion Time = 11TAD = 11 * 1.3 us
= **14.66 us**

Selected A/D Acquisition Time Select bits **ACQT2:ACQT0 = 001 i.e n= 2**
Derived A/D acquisition time = n * TAD = 2 * TAD
= 2 * 1.33 us
= 2.67 us
Which is greater than **calculated acquisition time = 2.4 us**

Effective conversion time = acquisition time + conversion time
= 14.66 us + 2.67 us
= 17.33 us

2.6 ADC operation

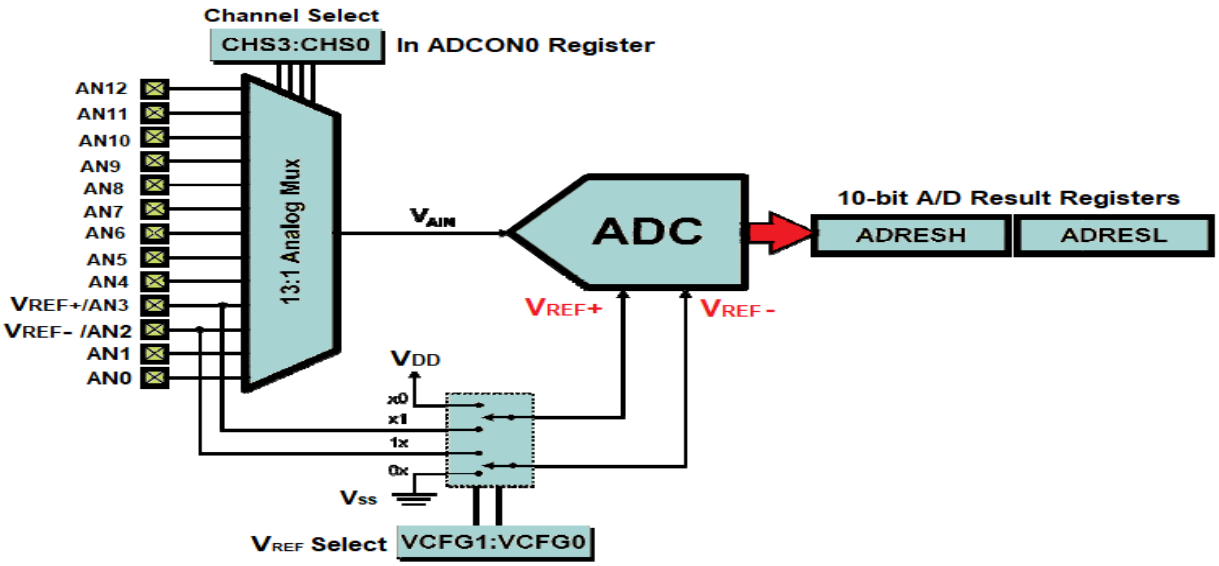


Figure 2 : Block Diagram of on-chip ADC Module

The following steps should be followed to perform an A/D conversion :

1. **Configure the A/D module:**

- Configure analog pins, voltage reference and digital I/O (ADCON1)
- Select A/D input channel (ADCON0)
- Select A/D acquisition time (ADCON2)
- Select A/D conversion clock (ADCON2)
- Turn on A/D module (ADCON0)

2. **Configure A/D interrupt (if desired):**
   - Clear ADIF bit
   - Set ADIE bit
   - Set GIE bit
3. **Wait the required acquisition time (if required).**
4. **Start conversion:**
   - Set GO/DONE bit (ADCON0 register)
5. **Wait for A/D conversion to complete, by either:**
   - Polling for the GO/DONE bit to be cleared OR
   - Waiting for the A/D interrupt
6. **Read A/D Result registers (ADRESH:ADRESL);**
7. **For next conversion, go to step 1 or step 2, as per bit is defined as TAD. A minimum wait of 3 TAD is acquisition starts.**

Pre-Test (Quiz):

1) What is the primary function of an Analog-to-Digital Converter (ADC)?
a: Convert digital signals into analog signals
b: Convert analog signals into digital signals
c: Store binary data in memory
d: Transmit signals wirelessly

2) Which factor determines the smallest change an ADC can detect?
a: Reference voltage
b: Conversion time
c: Resolution
d: Clock speed

3) How many steps does a 10-bit ADC have?
a: 256
b: 512
c: 1024
d: 2048

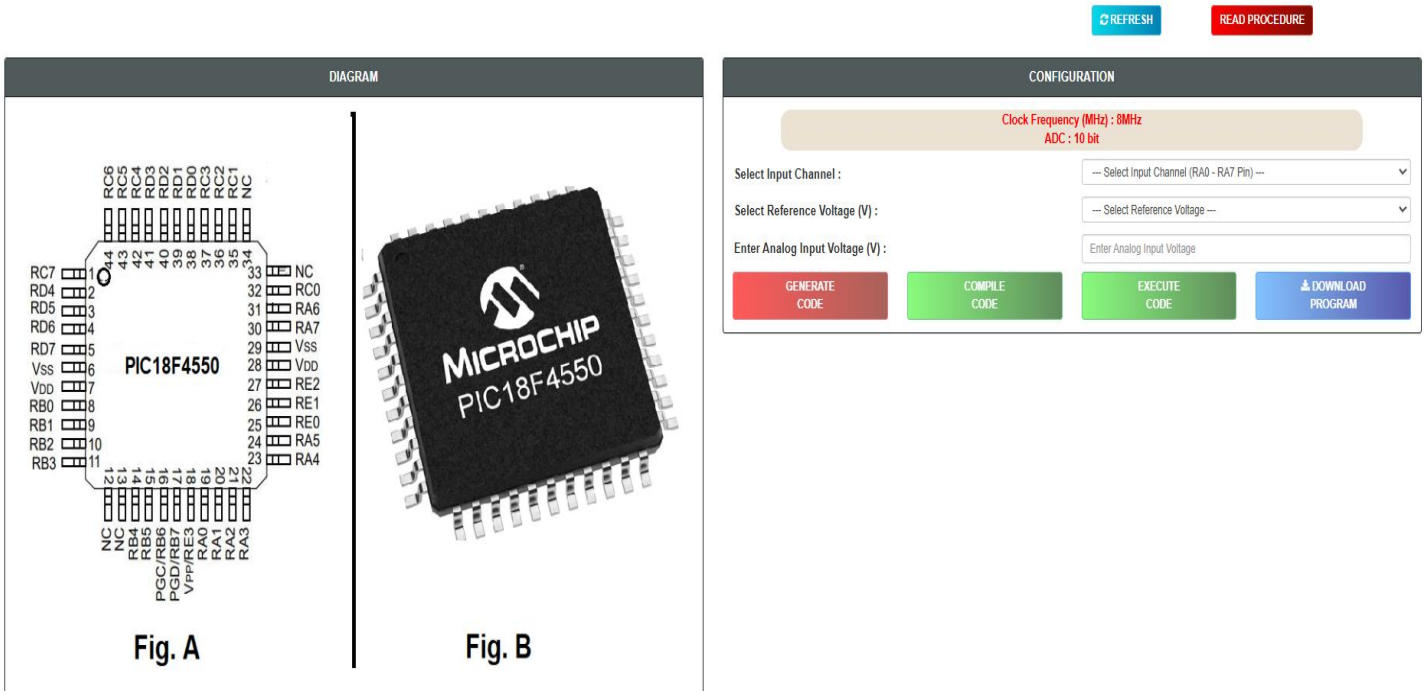
4) If an ADC has a reference voltage (Vref) of 5V and a resolution of 8 bits, what is the step size?
a: 5/256 = 19.53 mV
b: 5/1024 = 4.88 mV

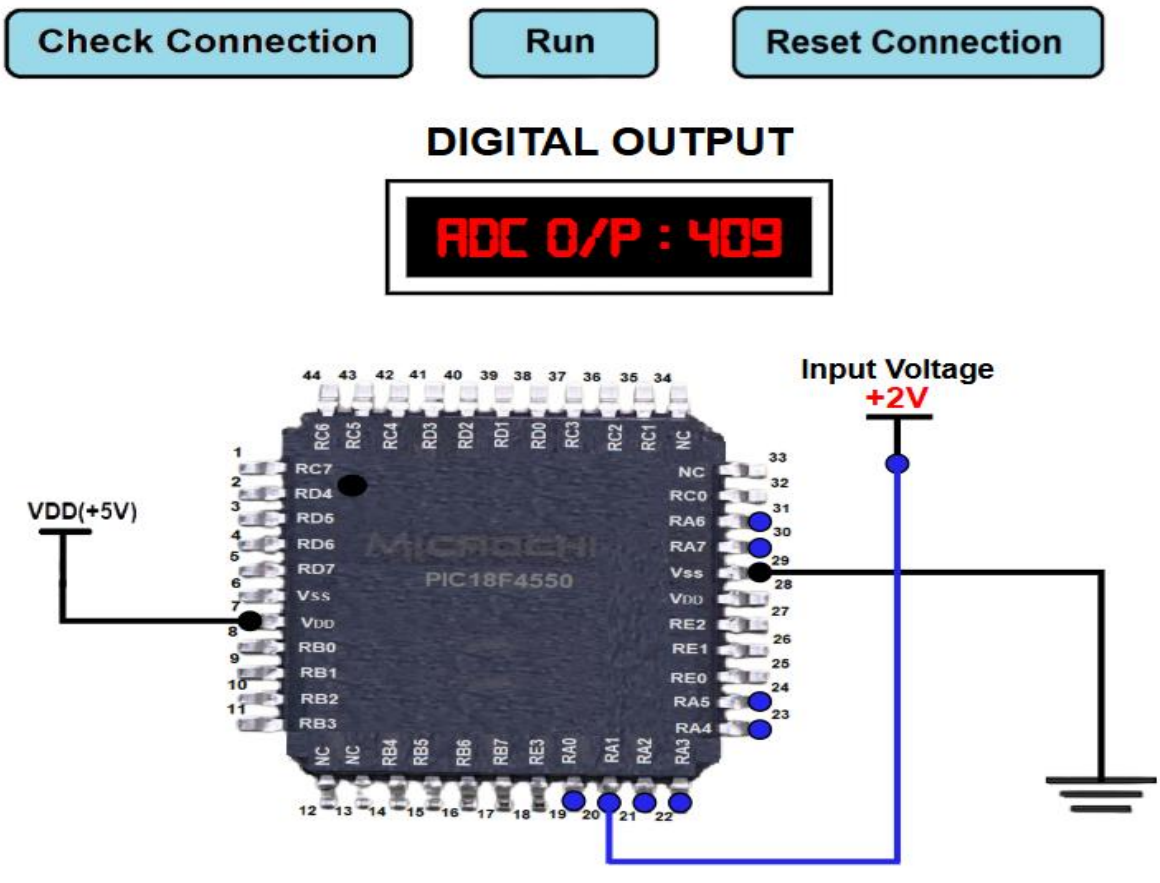
c: 5/4096 = 1.2 mV
d: 5/65536 = 0.076 mV

5) What does the ADCON0 register control in an ADC module?
a: Setting the reference voltage
b: Adjusting the acquisition time
c: Configuring digital I/O pins
d: Selecting the analog input channel and starting the conversion

Procedure
Simulator :

- step 1 : Select Input Channel (Pin RA0 to Pin RA7).
- step 2 : Select Reference Voltage.
- step 3 : Enter Analog Input Voltage in Volt.
- step 4 : Click on "GENERATE CODE" button to generate code.
- step 5 : Click on "COMPILE" button to compile code.
- step 6 : Click on "EXECUTE" button to execute code and goto mimic.
- step 7 : If you want to download code click on "DOWNLOAD" button.
- step 8 : The MIMIC will open on left side of screen.
- step 9 : Click on selected input channel terminal (blue circle) and click on analog input voltage terminal (blue circle) for connection of the selected input channel with analog input voltage or vice versa.
- step 10 : Click on "Check Connection" button.
- step 11 : If connection is wrong, click on "Reset Connection" button and follow steps 9 and 10.
- step 12 : If connection is successful, click on Run Button.
- step 13 : Now observe the ADC output on Digital Output Display.
- step 14 : Click on "REFRESH" button to re-executing the experiment.





Source code:

```
#include<pic18fregs.h>
#include<delay.h>
#pragma config XINST=OFF
#pragma config FOSC = INTOSCIO_EC
```

```
#pragma config WDT = OFF
#pragma config LVP = OFF
#define VREF 5
#define LED_TRIS_Output TRISDbits
#define LED_LAT_Output LATDbits
void ADC_init(void);
int ADC_read(int channel);
void ADC_Init()
{
   ADCON0=00000100;     //A/D Module is OFF and Channel 1 is selected
  TRISA = 0xFF; /* Set as input port */
  ADCON1 = 0x0E; /* Ref vtg is VDD and Configure pin as analog pin */
  ADCON2 = 0x92; /* Right Justified, 4Tad and Fosc/32. */
  ADRESH=0; /* Flush ADC output Register */
  ADRESL=0;
}
int ADC_Read(int channel)
{
  int digital;
  ADCON0 = (ADCON0 & 0b11000011) | ( (channel << 2) & 0b00111100 );
  ADCON0 |= ((1 << ADCON0bits.ADON) | (1 << ADCON0bits.GO));
  while(ADCON0bits.GO_NOT_DONE==1);
  digital = (ADRESH*256) | (ADRESL);
  return(digital);
}
void main(void)
{
float voltage;
  OSCCON=0x72; /* set internal clock to 8MHz */
  ADC_Init();
  TRISD = 0;
  while(1)
  {
  LATD= ADC_Read(0)*((float)VREF/(float)1023);
  } }
```

Conclusion: We have understood the Programming for On-Chip ADC using Virtual Lab.

Experiment #13 Write a program to control an LED brightness by generating a PWM signal on PIC18 series microcontroller

Aim

Write a program to control an LED brightness by generating a PWM signal on PIC18 series microcontroller

Objective

- To understand the working of PWM
- To study the on-chip CCP Module (PWM section) of PIC Microcontroller
- To interface DC motor, via Driver IC L293D, to PIC Microcontroller
- Apply the PWM signal to control the speed of DC Motor

Theory

1. PWM Technique

Pulse Width Modulation (PWM) is a technique used to control the power delivered to electrical devices by varying the duty cycle of a signal. It works by switching a signal between high and low states at a constant frequency, where the proportion of time the signal remains high (the duty cycle) determines the effective output power. A higher duty cycle means more power is delivered, while a lower duty cycle reduces power. PWM is widely used in motor speed control, LED dimming, and power regulation in electronic circuits due to its efficiency and precision.

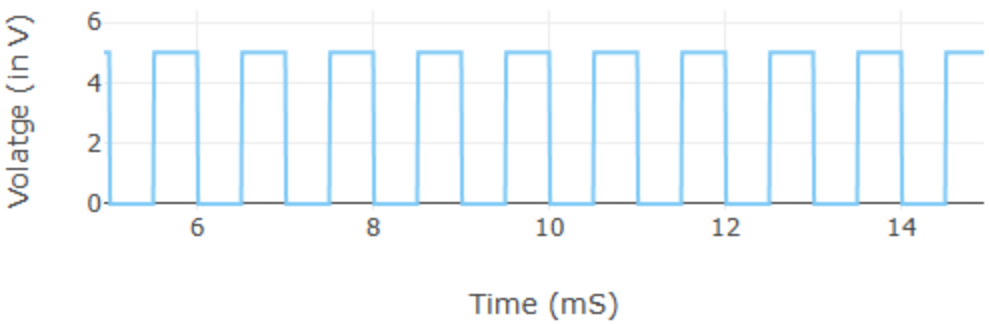


Fig: PWM output with 50% duty cycle

2. CCP Module of PIC18xx series microcontroller

Depending upon the device selected there are 1 – 4 CCP (Capture/Compare/PWM) modules in PIC micro controller. Each module contains a 16-bit register, which can operate as a 16-bit Capture register, a 16-bit Compare register or a PWM Master/Slave Duty Cycle register. Each Capture/Compare/PWM module is associated with a control register (generically, CCPxCON) and a data register (CCPRx). The data register, in turn, is comprised of two 8-bit registers: CCPRxL (low byte) and CCPRxH (high byte). All registers are both readable and writable. The CCP modules utilize Timers 1, 2 or 3, depending on the mode selected. Timer1 and Timer3 are available to modules in Capture or Compare modes, while Timer2 is available for modules in PWM mode.

The pin assignment for CCP2 (Capture input, Compare and PWM output) can change, based on device configuration. The CCP2MX Configuration bit determines which pin CCP2 is multiplexed to. By default, it is assigned to RC1 (CCP2MX = 1). If the Configuration bit is cleared, CCP2 is multiplexed with RB3. Changing the pin assignment of CCP2 does not automatically change any requirements for configuring the port pin. Users must always verify that the appropriate TRIS register is configured correctly for CCP2 operation, regardless of where it is located.

2.1 Capture Mode Applications

- Event arrival time recording
- Period measurement

- Pulse-width measurement
- Interrupt generation
- Event counting
- Time reference
- Duty cycle measurement

2.2 Compare Mode Applications

- Generate …
    - Single pulse
    - Train of pulses
    - Periodic waveform
- Start ADC conversion
- Time reference

3. PWM Mode of CCP Module

In Pulse-Width Modulation (PWM) mode, the CCPx pin produces up to a 10- bit resolution PWM output. Since the CCP2 pin is multiplexed with a PORTB or PORTC data latch, the appropriate TRIS bit must be cleared to make the CCP2 pin an output. A PWM output has a time base (period) and a time that the output stays high (duty cycle). The frequency of the PWM is the inverse of the period (1/period).

The simplified block diagram of the CCP1 configured to generate PWM is shown in Figure 1. with example waveforms in Figure 2. The block diagram for CCP2 is similar to CCP1. It can be seen that Figure 1. contains CCPR1L, Timer 2, comparator and the PR2 register working together. Although not shown, Timer 2 is still driven from the on-chip oscillator, through its own prescaler. The comparator output drives an R–S flip-flop. When the value in PR2 equals Timer 2, then the comparator clears the timer and sets the flip-flop, whose output goes high. This is seen in Figure 2. This action sets the PWM period.

Having established the PWM period, let us consider how the pulse width is determined. A second Compare register arrangement is introduced to do this. This is made up of the CCPR1H register, plus a second comparator. As the logic of the diagram shows, every time this comparator finds equal input values, it resets the output flip-flop, clearing the output to zero. It is this comparator that determines the pulse width. Again, this is shown in Figure 2. To change the pulse width, the programmer writes to the CCPR1L register, which acts as a buffer. Its value is transferred to CCPR1H only when a PWM cycle is complete, to avoid output errors in the process. The block diagram is made more complex because three of the registers are ‘stretched’, to make them potentially 10-bit instead of 8-bit. This increases the resolution. CCPR1L uses two bits of the CCP1CON register. CCPR1H is extended with an internal 2-bit latch, while the extension to Timer 2 is as described in Note 1 of Figure 1. Because of these two extra bits, in its 10-bit version it is effectively clocked direct from the internal oscillator signal, undivided. If the prescaler is used, then it acts on this frequency, not the usual Fosc/4. Notice, however, that the PR2 register remains at eight bits. This means that the PWM period has only an 8-bit equivalent resolution.

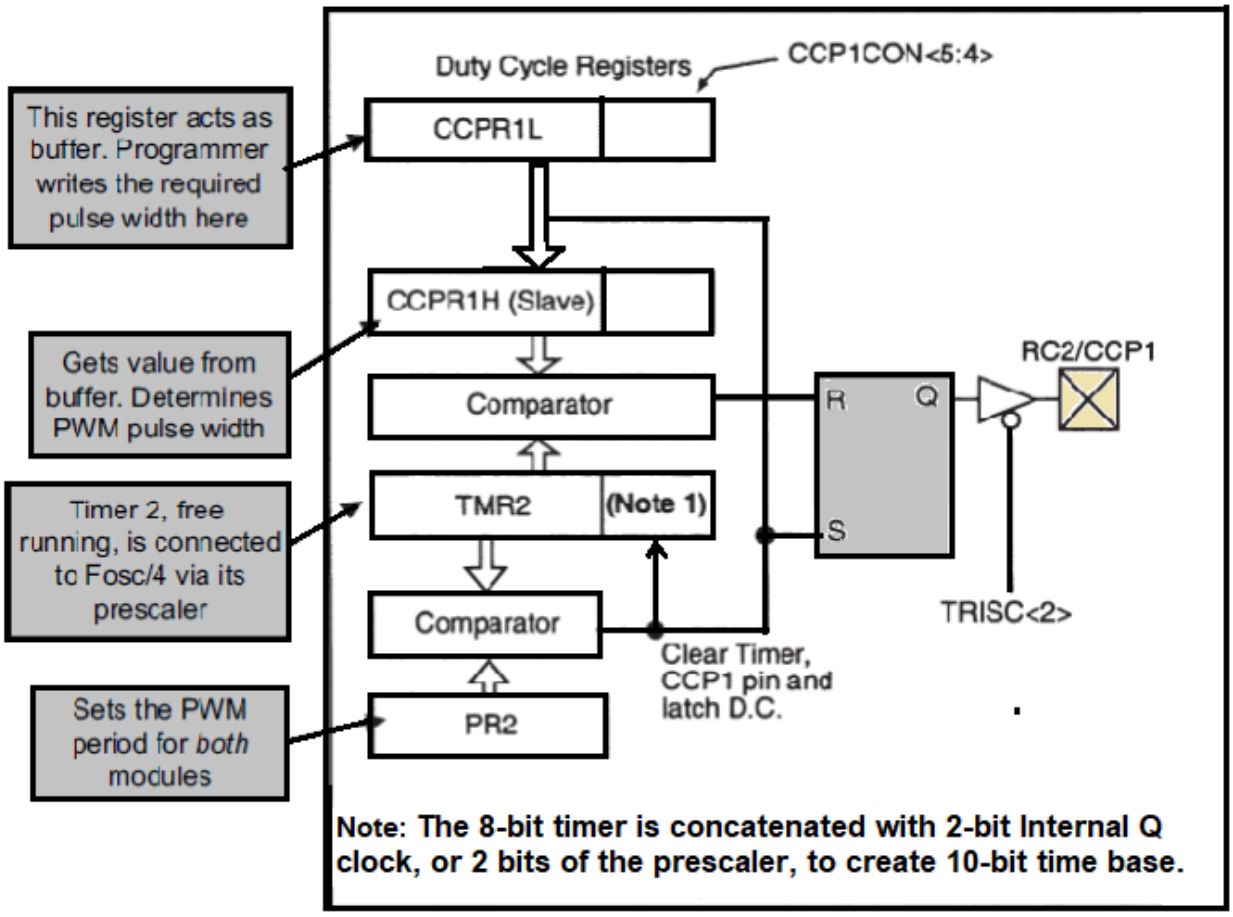


Figure 1 : PWM Block Diagram

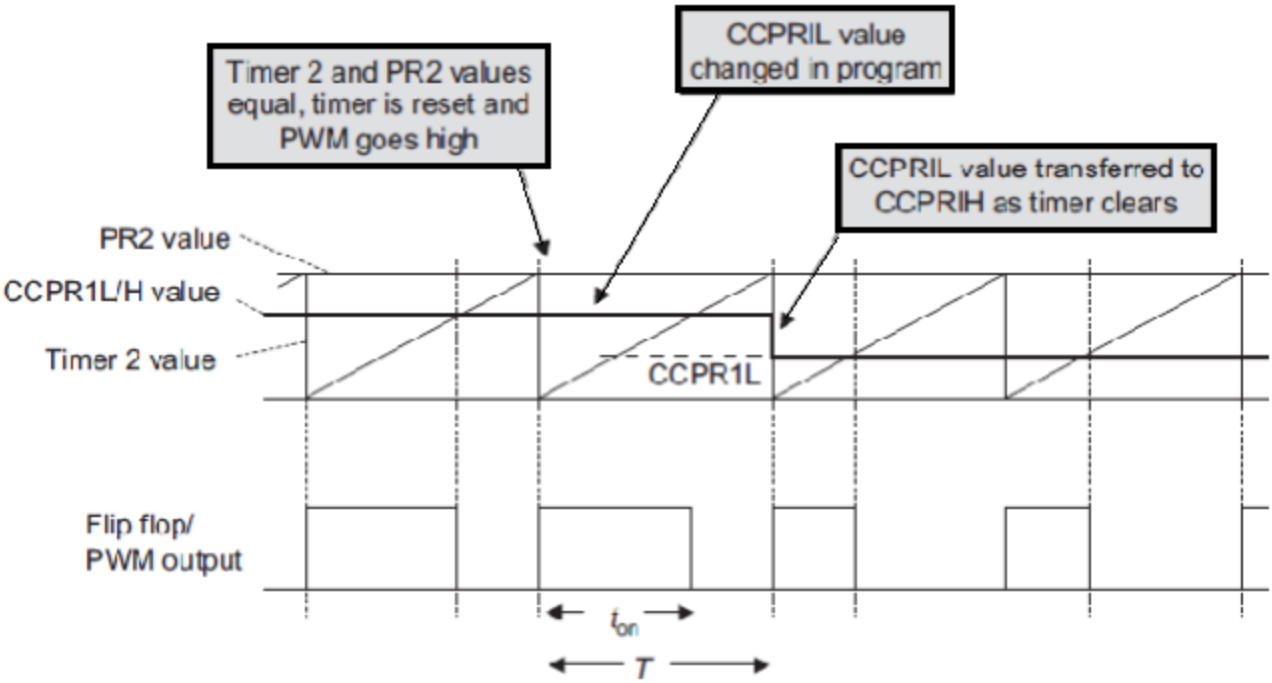


Figure 2 : PWM output waveform

3.1 PWM Period and Duty Cycle

The PWM period T is determined by the interaction of the PR2 register and the eight bits of Timer 2. It may be calculated as follows:

PWM Period = T_{pwm} = (PR2 + 1) x (Timer 2 input Clock)

PWM Period = T_{pwm} = (PR2 + 1) x (4 * Tosc * TMR2 Prescaler value)

Therefore to find the PR2 for desired PWM Period

$$PR2 = \left\{\left(\frac{\text{Tpwm}}{4 * Tosc * (TMR2\ Prescaler\ Value)}\right) - 1\right\}$$

Above equation can be represented in terms of frequency as below.

$$PR2 = \left\{\left(\frac{Fosc}{4 * Fpwm * (TMR2\ Prescaler\ Value)}\right) - 1\right\}$$

Where,
Timer 2 input Clock = 4 * Tosc * TMR2 Prescaler value
Fosc = Input Crystal Oscillator Frequency
Tosc = input clock period = 1/Fosc
Fpwm = PWM frequency
Tpwm = PWM Period = 1/Fpwm
The PWM pulse width Ton is determined by the interaction of the extended CCPR1H register (all 10 bits of it) and the extended (10-bit) Timer2. It may be calculated as follows:
Ton = (Pulse width register) x (PWM Timer input clock period)
Where, 'PWM timer input clock period' is the period of the clock input to the extended Timer 2 and 'pulse width register' is the value in the extended CCPR1L register.
Hence,
Ton = (CCPR1L: CCP1CON<5:4>) x (Tosc * TMR2 Prescaler value)
Note that there is not here a factor of four with the Tosc term, as we are using 10-bits of Timer 2. Rearranging the above equation, to find value of pulse width register (CCPR1L: CCP1CON <5:4>).

$$CCPR1L{:}\ CCP1CON < 5{:}4 > = \left\{\left(\frac{Ton}{Tosc * (TMR2\ Prescaler\ Value)}\right)\right\}$$

Above equation can be represented in terms of frequency and percentage Duty cycle (%DCpwm) as below.

$$CCPR1L{:}\ CCP1CON < 5{:}4 > = \left\{\left(\frac{\%DCpwm * Fosc}{100 * Fpwm * (TMR2\ Prescaler\ Value)}\right)\right\}$$

3.2 Pin Description

Signal	Pin No.	Symbol
CCP2^(1)	16	RC1/T1OSI/CCP2^(1)/UOE
CCP1	17	RC2/CCP1/P1A
CCP2^(1)	36	RB3/AN9/CCP2^(1)/VPO

Note 1: RB3 is the alternate pin for CCP2 multiplexing.

3.3 PWM Register Map:

SFR	Description	Access	Reset Value	Address
CCP1CON	Standard CCP1 Control Reg.	Read/Write	0x00	0xFBD

SFR	Description	Access	Reset Value	Address
CCPR1L	CCP1 Register low byte	Read/Write	unknown	0xFBE
CCPR1H	CCP1 Register high byte	No access	unknown	0xFBF
CCP2CON	Standard CCP2 Control Reg.	Read/Write	0x00	0xFBA
CCPR2L	CCP2 Register high byte	Read/Write	unknown	0xFBB
CCPR2H	CCP2 Register high byte	No access	unknown	0XFBC
T2CON	Timer 2 Control Register	Read/Write	0x00	0xFCA
TMR2	Timer 2 Register	Read/Write	0x00	0xFCC
PR2	Timer 2 Period register	Read/Write	0xFF	0xFCB

3.4 PWM Mode Register Description

3.4.1 Standard CPPx Control Register (CCPxCON – CCP1CON/CCP2CON)

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
_(1)	_(1)	DCxB1	DCxB0	CCPxM3	CCPxM2	CCPxM1	CCPxM0
bit 7							bit 0

Bit No.	Control Bit	Description
Bit 7 - 6	Unimplemented	Read as '0'
Bit 5 - 4	DCxB1:DCxB0	PWM Duty Cycle Bit 1 and Bit 0 for CCPx Module Unused in Capture mode & Compare mode: PWM mode: These bits are the two LSbs (bit 1 and bit0) of the 10-bit PWM duty cycle. The eight MSbs of the duty cycle are found in CCPR1L.
Bit 3 - 0	CCPxM3:CCPxM0	CCPx Module Mode Select bits 0000 = CCP disabled (resets CCPx module) 0001 = Reserved 0010 = Compare mode: toggle output on match (CCPxIF bit is set) 0011 = Reserved 0100 = Capture mode: every falling edge 0101 = Capture mode: every rising edge 0110 = Capture mode: every 4th rising edge 0111 = Capture mode: every 16th rising edge 1000 = Compare mode: initialize CCPx pin low; on compare match, force CCPx pin high (CCPxIF bit is set) 1001 = Compare mode: initialize CCPx pin high; on compare match, force CCPx pin low (CCPxIF bit is set) 1010 = Compare mode: generate software interrupt on compare match (CCPxIF bit is set, CCPx pin reflects I/O state) 1011 = Compare mode: trigger special event, reset timer, start A/D conversion on CCP2 match (CCPxIF bit is set) **11xx = PWM mode**

3.4.2 Timer 2 Control Register

U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0

U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
-	T2OUTPS3	T2OUTPS2	T2OUTPS1	T2OUTPS0	TMR2ON	T2CKPS1	T2CKPS0
bit 7							bit 0

Bit No.	Control Bit	Description
Bit 6 - 3	T2OUTPS3:T2OUTPS0:	Timer2 Output Postscaler Select bits (1:1 to 1:16) 0000 = 1:1 Postscaler 0001 = 1:2 Postscale . . 1111 = 1:16 Postscaler
Bit 2	TMR2ON:	Timer2 On bit 1 = Timer2 is on, 0 = Timer2 is off
Bit 1 - 0	T2CKPS1:T2CKPS0	Timer2 Clock Prescaler Select bits 00 = Prescaler is 1:1 01 = Prescaler is 1:4 1x = Prescaler is 1:16

4. Timer 2 Module

The Timer2 module timer incorporates the following features:

- 8-bit timer and period registers (TMR2 and PR2, respectively)
- Readable and writable (both registers)
- Software programmable prescaler (1:1, 1:4 and 1:16)
- Software programmable Postscaler (1:1 through 1:16)
- Interrupt on TMR2 to PR2 match
- Optional use as the shift clock for the MSSP module The module is controlled through the T2CON register which enables or disables the timer and configures the prescaler and Postscaler. Timer2 can be shut off by clearing control bit, TMR2ON (T2CON<2>), to minimize power consumption. A simplified block diagram of the module is shown in Figure 3.

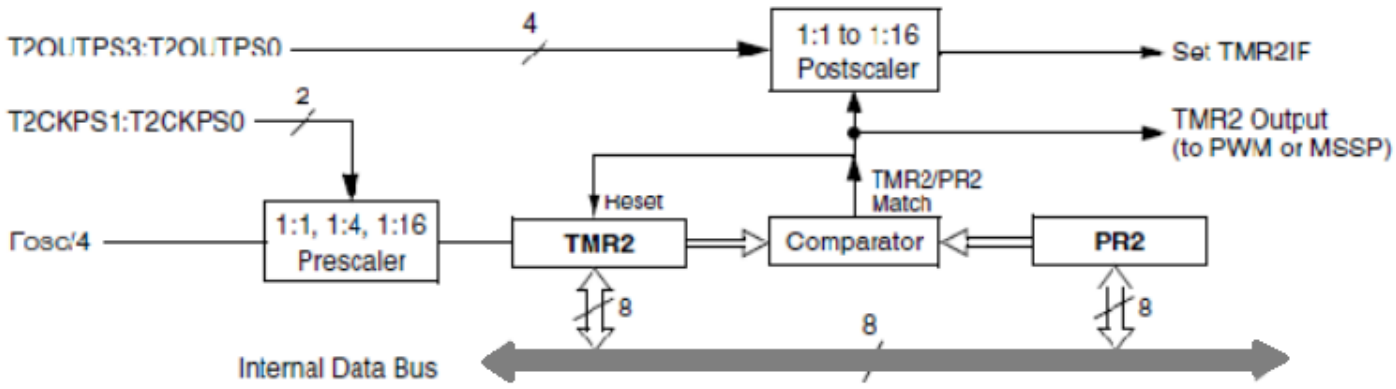


Figure 3. Simplified Block Diagram of Timer2 Module

4.1 Timer2 Operation

In normal operation, TMR2 is incremented from 00h on each clock (FOSC/4). A 2-bit counter/prescaler on the clock input gives direct input, divide-by-4 and divide-by- 16 prescale options. These are selected by the

prescaler control bits, T2CKPS1:T2CKPS0 (T2CON <1:0>). The value of TMR2 is compared to that of the period register, PR2, on each clock cycle. When the two values match, the comparator generates a match signal as the timer output. This signal also resets the value of TMR2 to 00h on the next cycle and drives the output counter/ Postscaler.

The TMR2 and PR2 registers are both directly readable and writable. The TMR2 register is cleared on any device Reset, while the PR2 register initializes at FFh. TMR2 is not cleared when T2CON is written. Both the prescaler and postscaler counters are cleared on the following events:

- a write to the TMR2 register
- a write to the T2CON register
- any device Reset (Power-on Reset, MCLR Reset, WDT Reset or Brown-out)

4.2 Timer2 Interrupt

Timer2 also can generate an optional device interrupt. The Timer2 output signal (TMR2 to PR2 match) provides the input for the 4-bit output counter/postscaler. This counter generates the TMR2 match interrupt flag which is latched in TMR2IF (PIR1<1>). The interrupt is enabled by setting the TMR2 Match Interrupt Enable bit, TMR2IE (PIE1<1>). A range of 16 postscale options (from 1:1 through 1:16 inclusive) can be selected with the postscaler control bits, T2OUTPS3:T2OUTPS0 (T2CON<6:3>).

4.3 TMR2 Output

The unscaled output of TMR2 is available primarily to the CCP modules, where it is used as a time base for operations in PWM mode. Timer2 can be optionally used as the shift clock source for the MSSP module operating in SPI mode.

5. Calculation for PWM operation

Desired PWM Frequency = 4 KHz

Therefore,

$$\text{PWM Period} = \frac{1}{\text{PWM Frequency}} = \frac{1}{4 \text{ X } 10^3} = 250 \text{ X } 10^{-6}$$

PWM period = 250 us

a. For Fosc = 20 MHz

$$\text{Tosc} = \frac{1}{\text{Fosc}} = \frac{1}{20 \text{ x} 10^6}$$

Tosc= 0.05 us

TMR2 Prescaler Value = 16,

$$\text{PR2} = \left\{\left(\frac{\text{PWM Period}}{4 * \text{Tosc} * (\text{TMR2 Prescale Value})}\right) - 1\right\}$$

$$\text{PR2} = \left\{\left(\frac{250 \text{ x } 10^{-6}}{4 \text{ x } 0.05 \text{ x } 10^{-6} \text{ x } 16}\right) - 1\right\}$$

$$\text{PR2} = \left\{\left(\frac{250 \text{ x } 10^{-6}}{3.2 \text{ x } 10^{-6}}\right) - 1\right\}$$

$$\mathbf{PR2 = \{78 - 1\} = \{77\} = 0x4E}$$

Calculate the Duty Cycle (CCPRXL:CCPCON<5:4>) 20%, 40%, 60% and 80% for the PWM Frequency of 4KHz,

$$\boldsymbol{CCPR1L:CCP1CON} < 5:4 > = \left\{\left(\frac{\%\boldsymbol{DCpwm} * \boldsymbol{Fosc}}{\mathbf{100} * \boldsymbol{Fpwm} * (\boldsymbol{TMR2\ Prescaler\ Value})}\right)\right\}$$

- Duty Cycle **20%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{20 * \ 20\text{x}10^6}{100 * 4\,x\ 10^3\ * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 62 = \ 0x3E \ \cdots \boxed{0000|0011|1110} \cdots \boxed{\boxed{0000|1111|}\boxed{|10|}}$$

$$\text{CCPR1L} = \mathbf{0x0F} \ \vdots \ \ \text{CCP1CON} < 5:4 > = \mathbf{10}$$

- Duty Cycle **40%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{40 * \ 20\text{x}10^6}{100 * 4\,x\ 10^3\ * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 125 = \ 0x7D \ \cdots \boxed{0000|0111|1101} \cdots \boxed{\boxed{0001|1111|}\boxed{|01|}}$$

$$\text{CCPR1L} = \mathbf{0x1F} \ \vdots \ \ \text{CCP1CON} < 5:4 > = \mathbf{01}$$

- Duty Cycle **60%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{60 * \ 20\text{x}10^6}{100 * 4\,x\ 10^3\ * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 187 = \ 0xBB \ \cdots \boxed{0000|1011|1011} \cdots \boxed{\boxed{0010|1110|}\boxed{|11|}}$$

$$\text{CCPR1L} = \mathbf{0x2E} \ \vdots \ \ \text{CCP1CON} < 5:4 > = \mathbf{11}$$

- Duty Cycle **80%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{80 * \ 20\text{x}10^6}{100 * 4\,x\ 10^3\ * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 250 = \ 0xFA \ \cdots \boxed{0000|1111|1010} \cdots \boxed{\boxed{0011|1110|}\boxed{|10|}}$$

$$\text{CCPR1L} = \mathbf{0x3E} \ \vdots \ \ \text{CCP1CON} < 5:4 > = \mathbf{10}$$

- Duty Cycle **100%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{100 * \ 20\text{x}10^6}{100 * 4\,x\ 10^3\ * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 312 = \ 0x138 \ \cdots \boxed{0001|0011|1000} \cdots \boxed{\boxed{0100|1110|}\boxed{|00|}}$$

$$\text{CCPR1L} = \mathbf{0x4E} \ \vdots \ \ \text{CCP1CON} < 5:4 > = \mathbf{00}$$

	Duty Cycle	**CCPR1L Value**	**CCP1CON<5:4> DC1B1:DC1B0**
Fosc = 20 MHz **PWM = 4 Khz** **PR2 = 0x4E**	20%	0b00001111 (0x0F)	0b10 (0x02)
	40%	0b00011111 (0x1F)	0b01 (0x01)
	60%	0b00101110 (0x2E)	0b11 (0x03)
	80%	0b00111110 (0x3E)	0b10 (0x02)
	100%	0b01001110 (0x4E)	0b00 (0x00)

b. For Fosc = 48 MHz

$$\text{Tosc} = \frac{1}{\text{Fosc}} = \frac{1}{48 \text{ x} 10^6}$$

Tosc= 20.83 ns
PWM period = 250 us
TMR2 Prescaler Value = 16,

$$\text{PR2} = \left\{\left(\frac{\text{PWM Period}}{4 * \text{Tosc} * (\text{TMR2 Prescale Value})}\right) - 1\right\}$$

$$\text{PR2} = \left\{\left(\frac{250 \text{ x } 10^{-6}}{4 \text{ x } 20.83 \text{ x } 10^{-9} \text{ x } 16}\right) - 1\right\}$$

$$\text{PR2} = \left\{\left(\frac{250 \text{ x } 10^{-6}}{1.33 \text{ x } 10^{-6}}\right) - 1\right\}$$

$$\mathbf{PR2} = \{\mathbf{187} - \mathbf{1}\} = \{\mathbf{186}\} = \mathbf{0xBA}$$

Calculate the Duty Cycle (CCPRXL:CCPCON<5:4>) 20%, 40%, 60% and 80% for the PWM Frequency of 4KHz,

$$\boldsymbol{CCPR1L}: \boldsymbol{CCP1CON} < 5:4 > = \left\{\left(\frac{\%\boldsymbol{DCpwm} * \boldsymbol{Fosc}}{\mathbf{100} * \boldsymbol{Fpwm} * (\boldsymbol{TMR2\ Prescaler\ Value})}\right)\right\}$$

- Duty Cycle **20%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{20 * 48\text{x}10^6}{100 * 4\,x\,10^3 * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 150 = 0x96 \cdots \boxed{0000|1001|0110} \cdots \boxed{0010|0101|\,|10|}$$

$$\text{CCPR1L} = \mathbf{0x25} \;:\; \text{CCP1CON} < 5:4 > = \mathbf{10}$$

- Duty Cycle **40%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{40 * 48\text{x}10^6}{100 * 4\,x\,10^3 * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 300 = 0x12C \cdots \boxed{0001|0010|1100} \cdots \boxed{0100|1011|\,|00|}$$

$$\text{CCPR1L} = \mathbf{0x4B} \;:\; \text{CCP1CON} < 5:4 > = \mathbf{00}$$

- Duty Cycle **60%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{60 * 48\text{x}10^6}{100 * 4\,x\,10^3 * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 450 = 0x1C2 \cdots \boxed{0001|1100|0010} \cdots \boxed{0111|0000|\,|10|}$$

$$\text{CCPR1L} = \mathbf{0x70} \;:\; \text{CCP1CON} < 5:4 > = \mathbf{10}$$

- Duty Cycle **80%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{80 * 48\text{x}10^6}{100 * 4\,x\,10^3 * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 600 = 0x258 \cdots \boxed{0010|0101|1000} \cdots \boxed{1001|0110|\,|00|}$$

$$\text{CCPR1L} = \mathbf{0x96} \;:\; \text{CCP1CON} < 5:4 > = \mathbf{00}$$

- Duty Cycle **100%**

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = \left\{\frac{100 * 48\text{x}10^6}{100 * 4\,x\,10^3 * 16}\right\}$$

$$(\text{CCPR1L: CCP1CON} < 5:4 >) = 750 = 0x2EE \cdots \boxed{0010|1110|1110} \cdots \boxed{1011|1011|\,|10|}$$

$$\text{CCPR1L} = \mathbf{0xBB} \;:\; \text{CCP1CON} < 5:4 > = \mathbf{10}$$

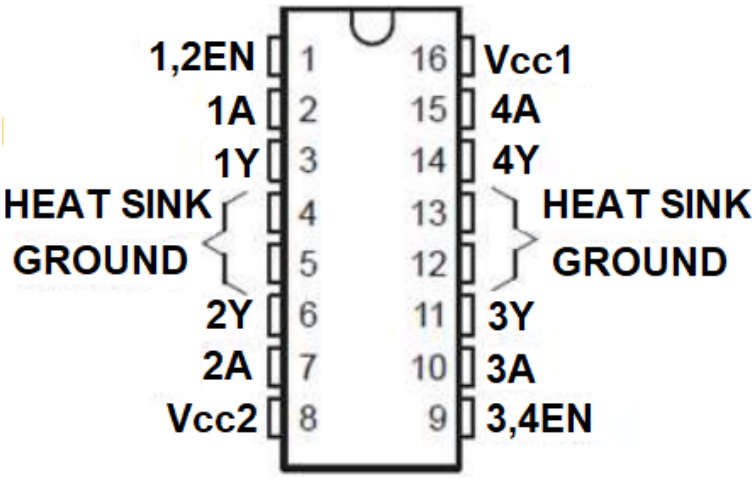
	Duty Cycle	**CCPR1L Value**	**CCP1CON<5:4> DC1B1:DC1B0**
Fosc = 48 MHz **PWM = 4 Khz** **PR2 = 0xBA**	20%	0b00100101 (0x25)	0b10 (0x02)
	40%	0b00011011 (0x4B)	0b00 (0x00)
	60%	0b01110000 (0x70)	0b10 (0x02)
	80%	0b10010110 (0x96)	0b00 (0x00)
	100%	0b10111011 (0xBB)	0B10 (0x02

6. Quadruple half H Drivers (L293D)

The L293D is quadruple high-current half-H drivers. The L293D is designed to provide bidirectional drive currents of up to 600-mA at voltages from 4.5 V to 36 V. The device is designed to drive inductive loads such as relays, solenoids, dc and bipolar stepping motors, as well as other high-current/high-voltage loads in positive-supply applications. All inputs are TTL compatible. Each output is a complete totem-pole drive circuit, with a Darlington transistor sink and a pseudo-Darlington source.

Drivers are enabled in pairs, with drivers 1 and 2 enabled by 1,2EN and drivers 3 and 4 enabled by 3,4EN as shown in pin diagram and function table. When an enable input is high, the associated drivers are enabled, and their outputs are active and in phase with their inputs. When the enable input is low, those drivers are disabled, and their outputs are off and in the high-impedance state. With the proper data inputs, each pair of drivers forms a full-H (or bridge) reversible drive suitable for solenoid or motor applications.

PIN DIAGRAM



FUNCTION TABLE
(each driver)

INPUTS†		OUTPUT
A	EN	Y
H	H	H
L	H	L
X	L	Z

Pre-Test(Quiz)

1) Which programming language is commonly used to write programs for PIC microcontrollers?
a: Python
b: Java
c: C/C++
d: JavaScript

2) Which microcontroller model was used in the experiment to generate the PWM signal for LED brightness control?
a: PIC16F877A
b: PIC18F4550
c: Arduino Uno
d: Raspberry Pi 3

3) What does PWM stand for in the context of microcontrollers?
a: Programmable Wide Modulation
b: Pulse Width Modulation
c: Power Wave Manipulation
d: Programmable Waveform Modulation

4) What is the purpose of generating a PWM signal in LED brightness control?
a: To toggle the LED on and off rapidly
b: To adjust the current flowing through the LED
c: To synchronize the LED with other devices

d: To change the LED colour

5) How is the LED brightness controlled using PWM?
a: By varying the voltage applied to the LED
b: By adjusting the frequency of the PWM signal
c: By changing the duty cycle of the PWM signal
d: By altering the LED's internal resistance

Procedure
Simulator :

- step 1 : Select Output Pin.
- step 2 : Select PWM Period value (mS).
- step 3 : Select Duty Cycle (10% - 100%).
- step 4 : Select LED Configuration.
- step 5 : Click on "GENERATE CODE" button to generate code.
- step 6 : Click on "COMPILE" button to compile code.
- step 7 : Click on "EXECUTE" button to execute code and goto mimic.
- step 8 : If you want to download code click on "DOWNLOAD" button.
- step 9 : The MIMIC will open on left side of screen.
- step 10 : Click on selected output pin terminal(red circle) and click on anode terminal(red circle) for connection of the selected output pin to anode or vice versa.
- step 11 : Click on cathode terminal(red circle) and click on resistor terminal(red circle) for connection of cathode to resister or vice versa.
- step 12 : Click on "Check Connection" button.
- step 13 : If connection is wrong, click on "Reset Connection" button and follow steps 10,11 and 12.
- step 14 : If connection is successful, click on "Run" Button.
- step 15 : Output LED is on. Observe the square wave graph.
- step 16 : Click on "REFRESH" button to re-executing the experiment.

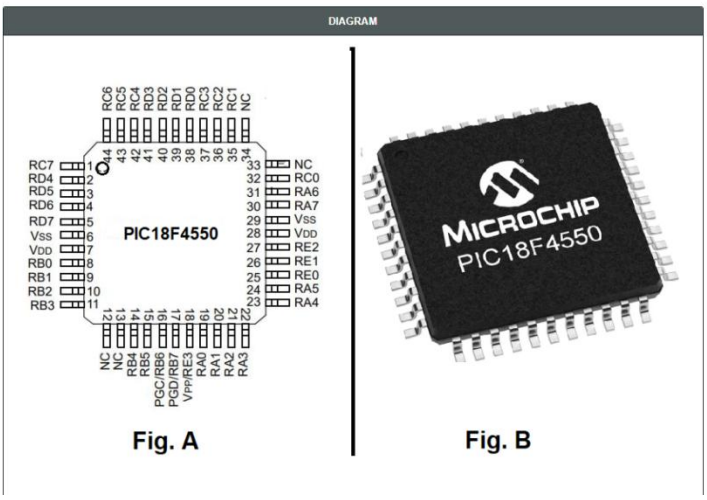
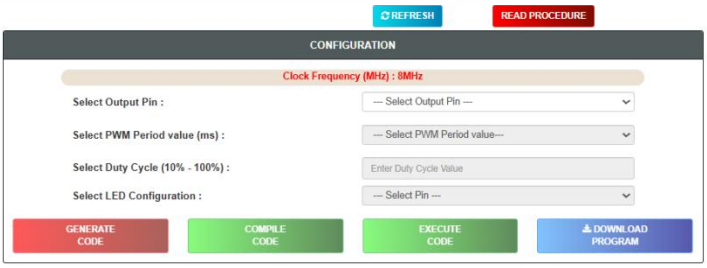
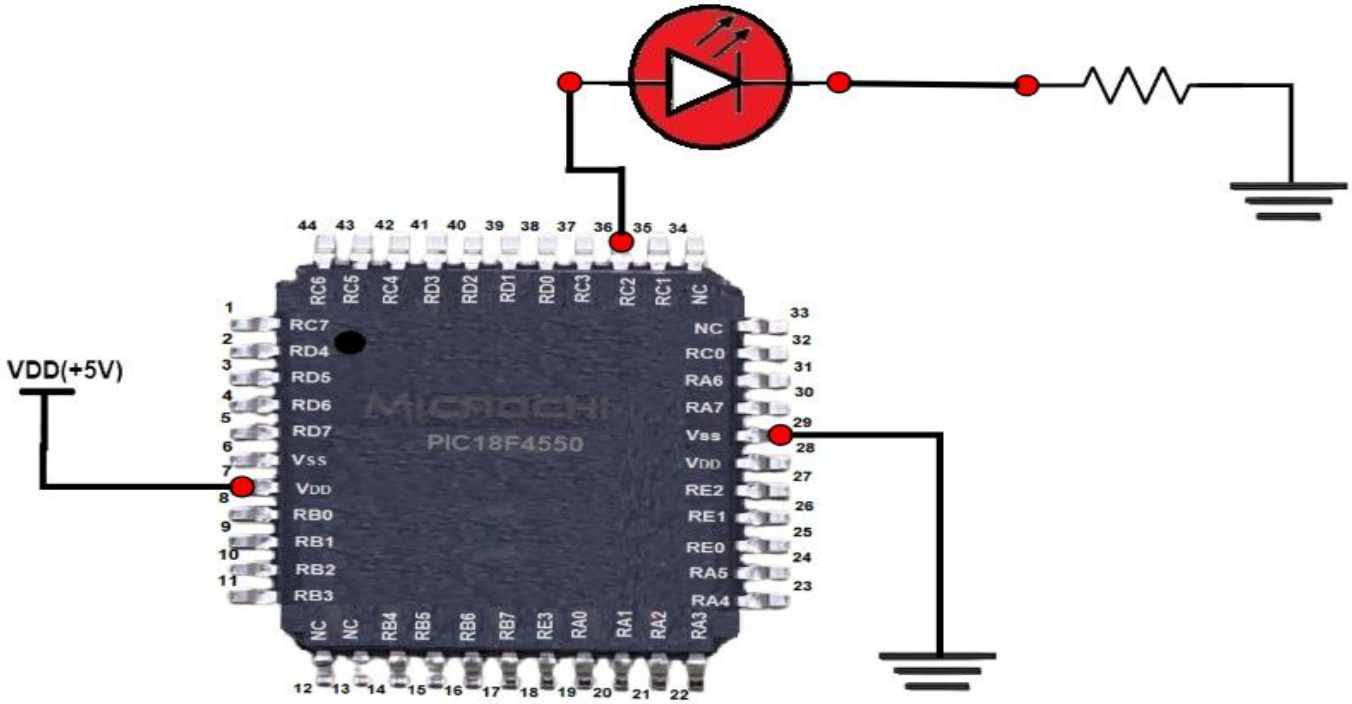


Fig. A Fig. B



Check Connection Run Reset Connection



Source Code:

```
#include<pic18fregs.h>
#include<delay.h>
```

```
#pragma config XINST=OFF
#pragma config FOSC = INTOSCIO_EC
#pragma config WDT = OFF
#pragma config LVP = OFF
void delay(unsigned int val);
void main(void)
{
OSCCON=0x72;    /* set internal clock to 8MHz */
TRISCbits.TRISC2=0;    /* Set CCP1 pin as output for PWM out */
PR2 = 249 ;
CCPR1L = 1;    /* load duty cycle */
T2CON=0;    /* no pre-scalar*/
CCP1CON=0x0C;    /* set PWM mode*/
TMR2=0;
T2CONbits.TMR2ON=1;    /* Turn ON Timer2 */
while(1)
{
CCPR1L = 125 ;   /* load duty cycle */
}
}
```

Conclusion:

We have understood the programming of controlling LED brightness by generating PWM signal on PIC 18 series microcontroller using virtual lab .

Expt #14 Programming for I/O interfacing (LED and Switch interfacing)

Aim

Write a program of LED blinking through push button using PIC18 series microcontroller

Objective

The objective of this experiment is to demonstrate the use of a PIC18 series microcontroller to control the blinking of an LED through a push button switch. The experiment aims to investigate the functionality of the microcontroller's GPIO (General Purpose Input Output) pins and how they can be programmed to control the state of an LED based on the state of a push button switch. By the end of the experiment, the participant should have a clear understanding of how to interface a push button switch with a microcontroller and use the microcontroller to control the blinking of an LED.

Theory

The PIC18 series microcontroller is a popular microcontroller family that is commonly used in various embedded systems applications due to its low power consumption and ease of programming. The GPIO (General Purpose Input Output) pins of a microcontroller are used to connect external devices such as LEDs, switches, sensors, etc. In this experiment, we will be using a push button switch to control the blinking of an LED connected to a GPIO pin of the PIC18 microcontroller.

When the push button switch is pressed, the input pin of the microcontroller is pulled high. The microcontroller's program then detects the change in state of the input pin and toggles the output pin connected to the LED, thereby turning it on. When the push button switch is released, the input pin returns to its initial state, and the microcontroller program detects this and turns off the LED.

To implement this functionality, we will use the programming language C. The program will be written to continuously monitor the state of the input pin and toggle the state of the output pin based on the state of the input pin.

Overall, this experiment will demonstrate the fundamental concept of interfacing a push button switch with a microcontroller and controlling an LED's blinking. This concept can be further extended to implement various applications such as controlling motors, sensors, and other electronic devices.

Now, for programming the inputs and outputs of PIC series microcontroller. For setting the peripheral to input and output, the necessary registers are PORTx, LATx, and TRISx.

Let's consider an example for PORT A:

PORTA is an 8-bit wide, bidirectional port. The corresponding Data Direction register is TRISA. Setting a TRISA bit (= 1) will make the corresponding PORTA pin an input (i.e., put the corresponding output driver in a high-impedance mode). Clearing a TRISA bit (= 0) will make the corresponding PORTA pin an output (i.e., put the contents of the output latch on the selected pin)

The Data Latch register (LATA) is also memory mapped. Read-modify-write operations on the LATA register read and write the latched output value for PORTA.

Schematic

Components used in simulation:

- Microcontroller: PIC18F4550
- Aqua color LED
- Resistor: 330 Ohms
- Pushbutton

In this Example schematic, the push button is connected to pin RC0 of PORT C and LED is connected across RD0 of the PORT D of the microcontroller.

Current Sinking Operation

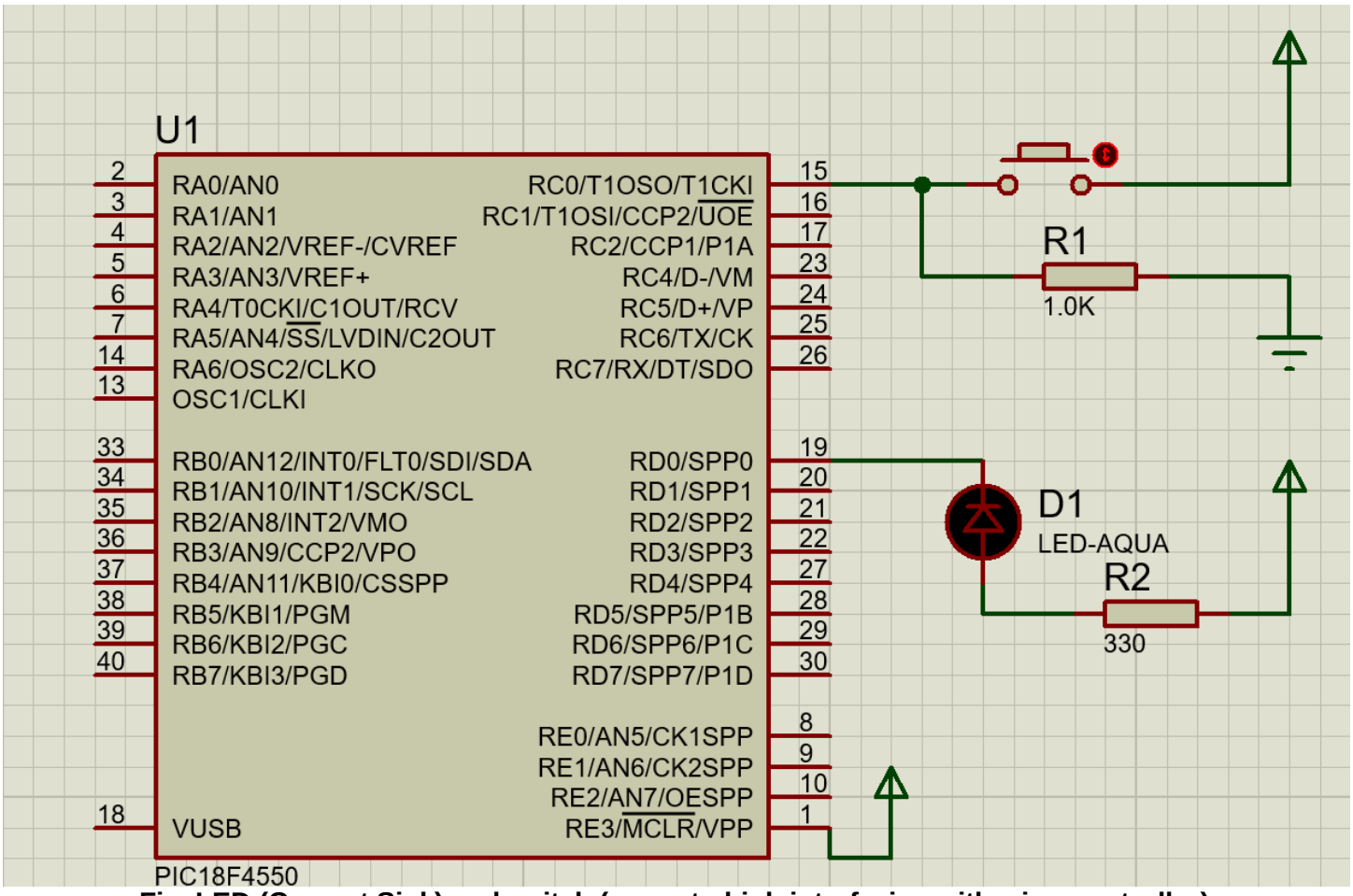


Fig: LED (Current Sink) and switch (press to high interfacing with microcontroller)

In the current Sinking Operation, Current flows into the device. In this method, the Button is PUSHED-TO-HIGH and LED starts blinking.

<u>Current Sourcing Operation</u>

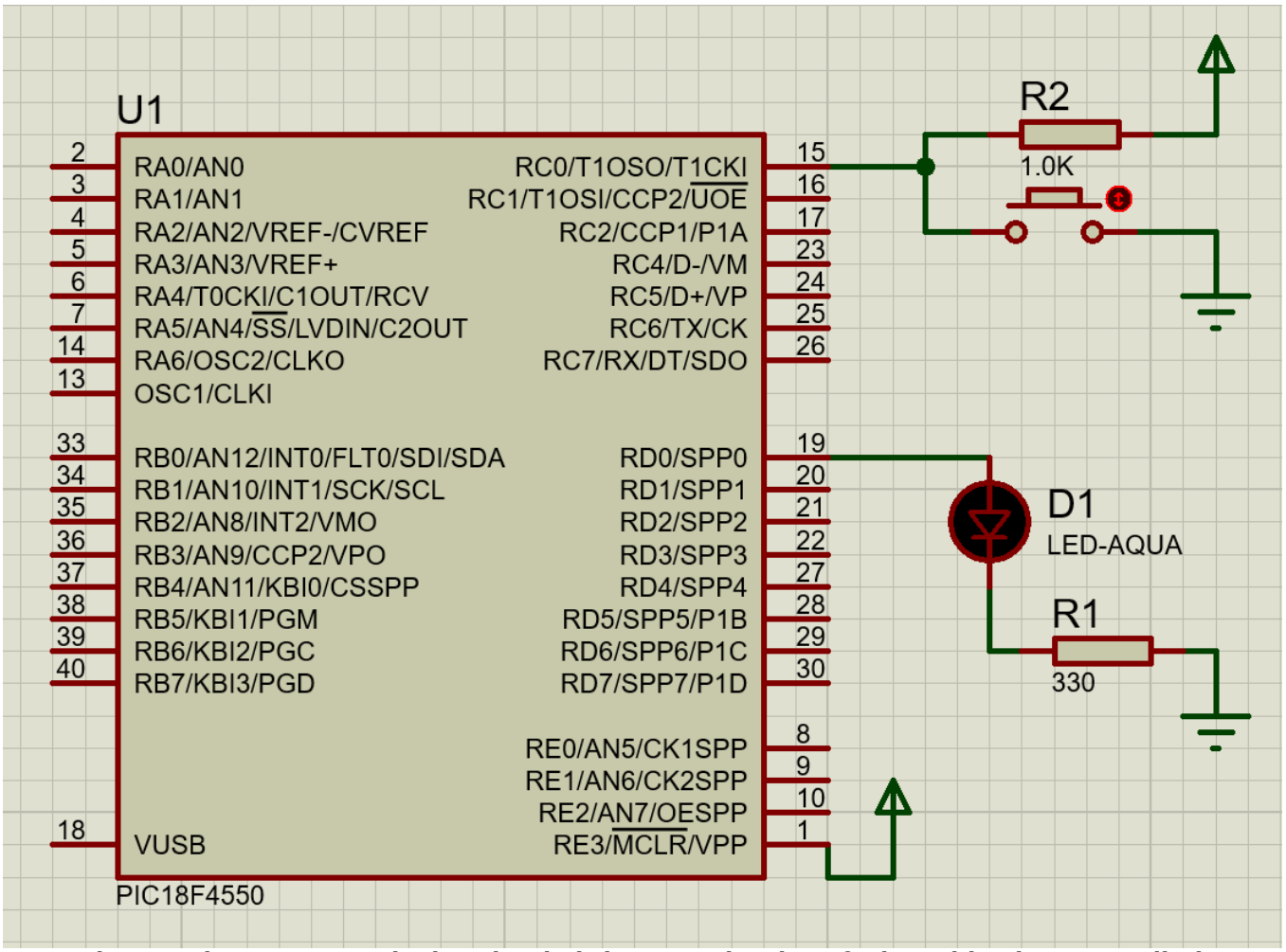


Fig: LED (current sourcing) and switch (press to low interfacing with microcontroller)

In the current Sourcing Operation, Current flows out of the device. In this method, the Button is PUSHED-TO-LOW and starts blinking.

Pre Test (Quiz)

1) What is an embedded system?

a: A system that can be connected to the internet
b: A system that is built into a larger device or machine
c: A system that requires no external components to function
d: A system that can only be programmed using assembly language

2) Which of the following should a microcontroller at-least should consist of?

a: CPU, ROM, I/O ports, and timers
b: RAM, ROM, I/O ports, and timers
c: CPU, RAM, I/O ports, and timers
d: CPU, RAM, ROM, I/O ports, and timers

3) Which of the following file extension that is loaded in a microcontroller for executing any instruction?

a: .c
b: .txt
c: .hex
d: .doc

4) Which of the following microcontroller doesn't match with its architecture below?

a: ARM7 – Von Neumann
b: Microchip PIC – Harvard
c: ARM9 – Harvard
d: MSP430 – Harvard

5) What is the difference between a microcontroller and a microprocessor?

a: A microcontroller has more processing power than a microprocessor
b: A microcontroller contains both memory and input/output peripherals, whereas a microprocessor does not
c: A microcontroller is used in personal computers, while a microprocessor is used in embedded systems
d: There is no difference between a microcontroller and a microprocessor

Procedure

Simulator :

- step 1 : Select Input Pin For Switch.
- step 2 : Select Output Pin For LED.
- step 3 : Select Switch Configuration.
- step 4 : Select LED Configuration.
- step 5 : Click on "GENERATE CODE" button to generate code.
- step 6 : Click on "COMPILE" button to compile code.
- step 7 : Click on "EXECUTE" button to execute code and goto mimic.
- step 8 : If you want to download code click on "DOWNLOAD" button.
- step 9 : The MIMIC will open on left side of screen.
- step 10 : Connect the Output Pin to the Anode.
- step 11 : Click on the cathode terminal (red circle) and then click on the resistor terminal (red circle) to connect the cathode with the resistor or vice versa.
- step 12 : Click on the selected input pin terminal (blue circle) and then click on the switch terminal (blue circle) to connect the input pin with the switch terminal or vice versa.
- step 13 : Click on "Check Connection" button.
- step 14 : If connection is wrong, click on "Reset Connection" button and follow steps 10, 11, 12 and 13.
- step 15 : If connection is successful, click on Switch available on the screen with black color and hold the switch for a few seconds.
- step 16 : While holding the switch, Output LED is on. Observe the square wave graph to see how the output changes until you complete the observation.
- step 17 : Click on "REFRESH" button to re-executing the experiment.

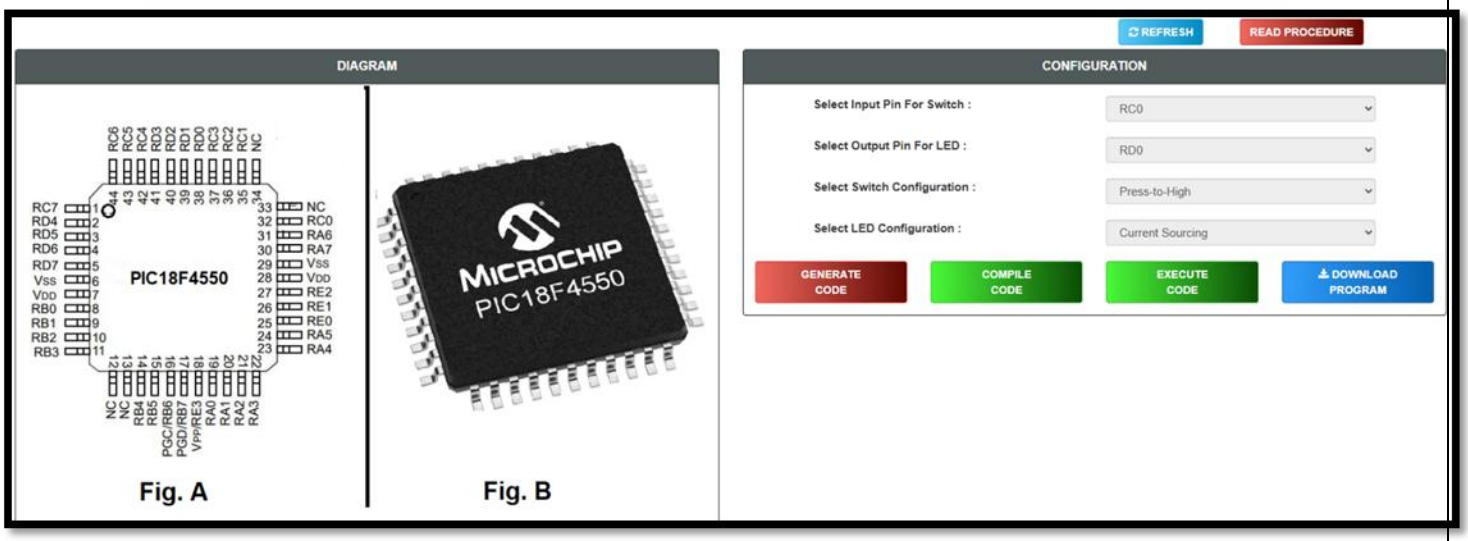
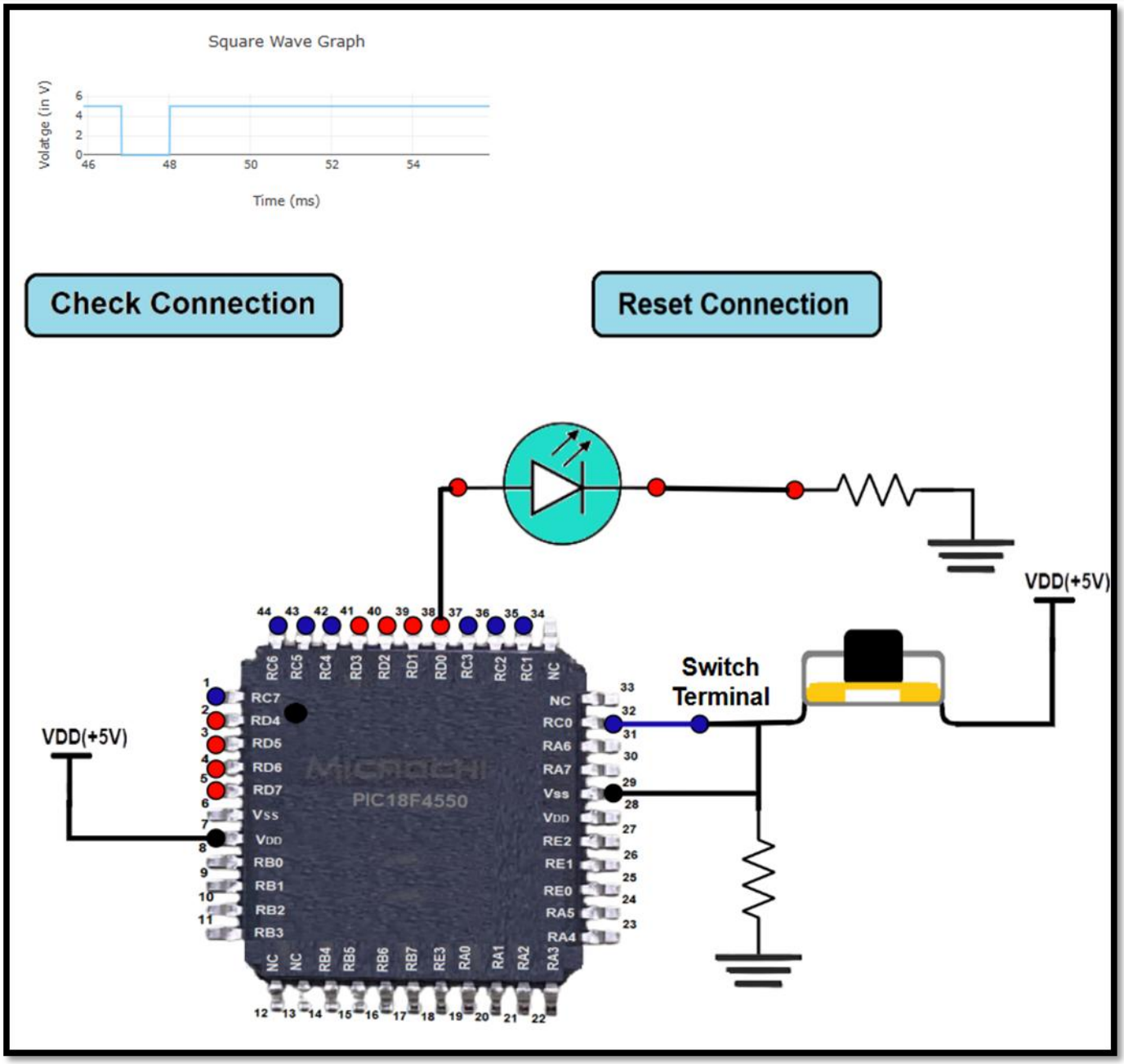


Fig. A

Fig. B



Source Code:

```
#include<pic18fregs.h>
#include<delay.h>
 #pragma config XINST=OFF
 #pragma config FOSC = INTOSCIO_EC
 #pragma config WDT = OFF
 #pragma config LVP = OFF
 #define LED_TRIS_Input TRISCbits.TRISC0
 #define LED_LAT_Input PORTCbits.RC0
 #define LED_TRIS_Output TRISDbits.TRISD0
 #define LED_LAT_Output LATDbits.LATD0
 void main(void)
 {
 LED_TRIS_Input=1;
 LED_READ_Input=0;
 LED_TRIS_Output=0;
 LED_LAT_Output=0;
 while(1)
 {
 if (LED_READ_Input == 0X01) // If the switch is pressed
 {
 LED_LAT_Output = 0X01; // LED ON
 }
 else
 {
 LED_LAT_Output = 0X00; // LED OFF
 }
 }
 }
```

Conclusion:

We have understood the Programming for I/O interfacing (LED and Switch interfacing) using Virtual Lab